

Course UEF31

Computer Architecture

Architecture des ordinateurs

- Chapter 1: General organization of the central unit
- Chapter 2: Inside the processor
- Chapter 3: Case study, the Intel 8086 processor
- Chapter 4: Case study, 32-bit and 64-bit processors
- Chapter 5: Modern processor architectures

Teacher: Dr. Benabderrezak Youcef

Coefficient: 3 **Credits:** 5

Assessment: exam 60%, continuous assessment 40%

How to use this course

This course is written for you, even if this is the first time you look inside a computer. We start with simple ideas. Step by step, we go up to real programs in assembly language. At the end, we understand how the processors in our phones and laptops work.

The coloured boxes

On every page you will find coloured boxes. Each colour has a job:

Definition

A new important word. We also give its French name, because you will meet it in the official syllabus and in many books.

Everyday analogy

A difficult idea, explained with something you know from daily life: a kitchen, a school, a bus...

Step by step

A calculation or an execution, done one small step at a time. We never skip a step.

Watch out

A common mistake, or a small detail that changes the result.

Did you know?

A fun extra fact. You do not need it for the exam, but it helps you understand.

The programs in this course

- Every program is complete, and comes with a table **"What does each line do?"**.
- The 8086 programs are written for the **EMU8086** emulator. The 64-bit programs are written for **NASM** on **Linux**.
- All the programs **were really run**. What you see in the black screen is the real output, and the register tables come from the real execution.

Watch out

Do not only read! Copy each program into EMU8086, run it one instruction at a time (the *Single step* button), and watch the registers change. This is how you learn this course.

Contents

How to use this course **1**

1 General Organization of the Central Unit **8**

1.1 What is a computer? . . . 8

1.1.1 Hardware and software . . . 9

1.1.2 The layers of a computer . . . 9

1.1.3 How does a program run? . . . 9

1.2 A short history of computers . . . 9

1.3 Quick reminder: how does a computer count? . . . 10

1.3.1 Memory size units . . . 12

1.4 The main parts of a computer . . . 12

1.5 The two basic models: Von Neumann and Harvard . . . 13

1.5.1 The Von Neumann model . . . 13

1.5.2 The Harvard model . . . 14

1.5.3 Comparison and uses . . . 14

1.6 The processor: role and parts . . . 15

1.6.1 The clock and the speed of the processor . . . 15

1.7 Memory . . . 15

1.7.1 What does memory look like inside? . . . 16

1.7.2 Only two operations: read and write . . . 16

1.7.3 Types of memory . . . 16

1.7.4 Calculating the size of a memory . . . 17

1.8 Buses . . . 18

1.8.1 Bus width . . . 19

1.9 Exercises . . . 19

2 Inside the Processor **21**

2.1 A look inside the processor . . . 21

2.2 Registers . . . 22

2.2.1 Why are registers important for speed? . . . 22

2.2.2 Types of registers . . . 23

2.2.3 The flags register . . . 23

2.2.4 The difference between CF and OF . . . 23
2.3 The arithmetic and logic unit (UAL) . . . 24
2.3.1 How do logic gates add? The half adder . . . 24
2.3.2 The full adder . . . 25
2.3.3 A 4-bit adder . . . 25
2.3.4 And subtraction? The same adder! . . . 26
2.3.5 Comparing by subtracting . . . 26
2.3.6 A real test: how do the flags change? . . . 26
2.4 The control unit (UC) . . . 27
2.4.1 Micro-operations, pulse by pulse . . . 28
2.4.2 Two kinds of control units . . . 28
2.5 The instruction set . . . 28
2.5.1 The shape of an instruction . . . 28
2.5.2 How many instructions can a processor have? . . . 29
2.5.3 Number of addresses in an instruction . . . 29
2.5.4 Classes of instructions . . . 30
2.5.5 CISC and RISC . . . 30
2.6 Addressing modes . . . 30
2.6.1 The basic modes . . . 30
2.6.2 When do we use each mode? . . . 31
2.6.3 The modes in a real program . . . 31
2.7 The steps of an instruction . . . 32
2.7.1 A complete example, step by step . . . 33
2.8 Exercises . . . 34

3 Case Study: the Intel 8086 Processor 35
3.1 The 80x86 family . . . 35
3.2 Inside the 8086 . . . 36
3.3 The 8086 registers . . . 36
3.3.1 General registers . . . 36
3.3.2 Pointer and index registers . . . 37
3.3.3 Segment registers . . . 37
3.3.4 Instruction pointer and flags . . . 38
3.4 Segmented addressing . . . 38
3.4.1 The problem . . . 38
3.4.2 The solution: segment and offset . . . 38

3.4.3 Why multiply by 16? . . . 39
3.4.4 Default pairs . . . 39
3.5 8086 addressing modes . . . 40
3.6 A first program . . . 41
3.6.1 The tools . . . 41
3.6.2 The program . . . 41
3.6.3 Defining variables . . . 42
3.7 How an instruction runs inside the 8086 . . . 43
3.8 The instruction set in detail . . . 43
3.8.1 Transfer instructions . . . 43
3.8.2 Arithmetic instructions . . . 44
3.8.3 Logic instructions . . . 46
3.8.4 Flow control: jumps . . . 48
3.8.5 Loops and repetition . . . 51
3.8.6 Generating machine code . . . 53
3.9 Input and output . . . 54
3.9.1 Ports: `in` and `out` . . . 55
3.9.2 DOS services: the `int 21h` interrupt . . . 55
3.10 The stack . . . 56
3.10.1 Why do we need a stack? . . . 57
3.11 Functions and procedures . . . 57
3.11.1 How does `ret` know where to go back? . . . 57
3.11.2 A procedure that prints a number in decimal . . . 57
3.11.3 Passing parameters on the stack . . . 59
3.12 Interrupts . . . 60
3.12.1 Kinds of interrupts . . . 60
3.12.2 The interrupt vector table . . . 60
3.12.3 What does the processor do on `int n`? . . . 60
3.12.4 A hardware interrupt, step by step . . . 61
3.12.5 A program: our own interrupt . . . 61
3.13 Exercises . . . 62

4 Case Study: 32-bit and 64-bit Processors 64
4.1 From 16 to 32, then 64 bits . . . 64
4.1.1 The main differences . . . 64
4.1.2 Why do we need more than 32 bits? . . . 65

4.2 How the architecture grew: from IA-32 to AMD64 . . . 65
4.2.1 The working modes of the processor . . . 65
4.3 Registers in 32-bit and 64-bit processors . . . 66
4.3.1 Nested registers . . . 66
4.3.2 A test: what happens to the other bits? . . . 67
4.4 Advanced addressing modes . . . 68
4.4.1 From segmented addressing to flat addressing . . . 68
4.4.2 The general form of an address . . . 69
4.4.3 RIP-relative addressing . . . 71
4.5 The instruction set . . . 71
4.5.1 The same instruction on 16, 32 and 64 bits . . . 71
4.5.2 Comparison: a 32-bit program and a 64-bit program . . . 72
4.5.3 What does the compiler write? From C to assembly . . . 73
4.5.4 SSE and AVX: computing several numbers at once . . . 74
4.6 The instruction cycle and the pipeline . . . 76
4.6.1 The five stages . . . 76
4.6.2 Without a pipeline and with a pipeline . . . 76
4.6.3 Hazards: when does the pipeline stop? . . . 77
4.6.4 Example: `mov`, `add` and `jmp` in 64 bits . . . 77
4.7 Exercises . . . 78

5 Modern Processor Architectures 80
5.1 Modern processors: trends and changes . . . 80
5.1.1 Fifty years of progress . . . 80
5.1.2 The frequency wall . . . 80
5.1.3 Today's trends . . . 81
5.2 Multi-core processors and parallelism . . . 81
5.2.1 The ideas . . . 81
5.2.2 Hyper-Threading: one core for two threads . . . 82
5.2.3 A test: the same work on 1, 2, 4 and 8 threads . . . 82
5.2.4 Amdahl's law: the limits of parallelism . . . 84
5.2.5 Benefits and challenges . . . 85
5.3 Cache memory and the memory hierarchy . . . 85
5.3.1 The problem: memory is very slow . . . 86
5.3.2 The memory hierarchy . . . 86
5.3.3 A test: we can see the memory levels . . . 86

5.3.4 The principle of locality: why does cache memory work? 88
5.3.5 A test: the order of reading changes everything 89
5.3.6 Hits, misses and average access time 90
5.3.7 Where do we put data in the cache? (mapping) 90
5.3.8 Replacement and writing 91
5.4 Exercises 91

A 8086 Instruction Summary **93**

B EMU8086 and NASM: differences in writing **95**

C Glossary **96**

Chapter 1

General Organization of the Central Unit

Summary: what will we learn in this chapter?

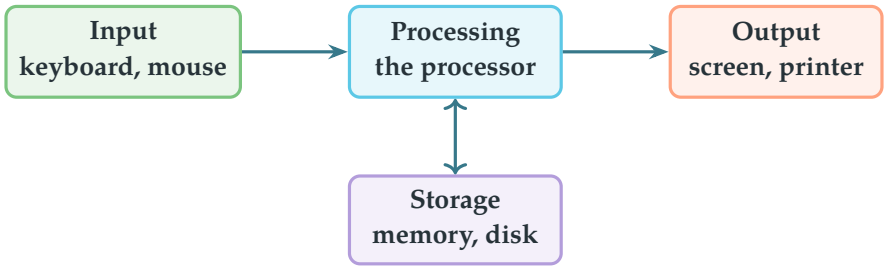
- What a computer is, and how it changed through history.
- The main parts: the processor, the memory, the input/output devices.
- The Von Neumann and Harvard models, and the difference between them.
- The types of memory, and how to calculate a memory size.
- The three buses: the data bus, the address bus, the control bus.

1.1 What is a computer?

Definition

A **computer** (Fr. *ordinateur*) is an electronic machine. It receives **data**, processes it with a **program** (a list of instructions), and gives **results**. It can also store data and results.

Everything a computer does fits in four verbs: it **receives**, it **processes**, it **stores**, it **outputs**.



Let us take a simple example: you use the calculator app on your computer to compute 25×4.

1. **Input**: you type 25, then $\times$, then 4 on the keyboard. Each key press becomes a number code that goes into the computer.
2. **Temporary storage**: the computer keeps the two numbers in memory, because it needs them soon.
3. **Processing**: the processor reads the two numbers, runs the multiply instruction, and gets 100.
4. **Output**: the computer sends the result to the screen, and you see it.

Every program you use, even a complex one (a game, a browser, a photo app), is in fact millions of these small steps, done very fast.

Everyday analogy

A computer is like a **restaurant kitchen**. The ingredients (vegetables, meat) are the **data**. The recipe is the **program**. The cook is the **processor**. The fridge is the **memory**. The finished dish is the **result**. The cook does not decide alone: he follows the recipe step by step.

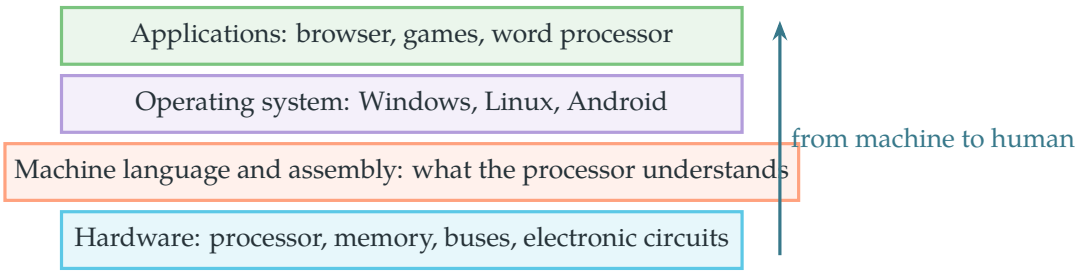
1.1.1 Hardware and software

- **Hardware** (Fr. *matériel*): everything you can touch: the processor, the memory, the screen, the wires.
- **Software** (Fr. *logiciel*): the programs that tell the hardware what to do: the operating system, the browser, games.

In this course we study **the hardware from the inside**, and we see how it runs programs.

1.1.2 The layers of a computer

We can see a computer as layers, one on top of the other. Each layer uses the services of the layer below it, without knowing all its details:



In this course we work on the **two lowest layers**: the hardware, and the assembly language that talks to it directly.

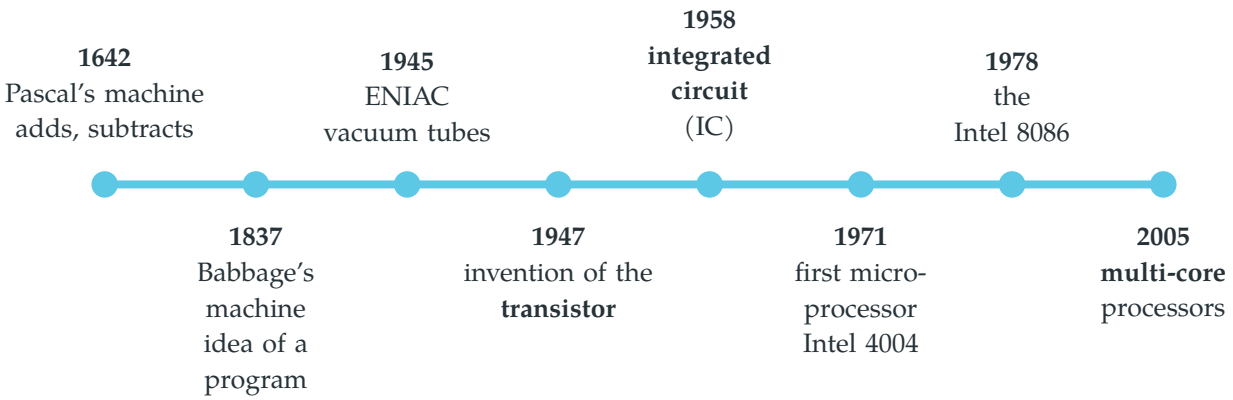
1.1.3 How does a program run?

When you double-click on a program icon, these steps happen:

1. The program is stored on the **disk** (SSD or HDD) as a file of zeros and ones.
2. The operating system **copies it into the main memory** (RAM), because the disk is very slow and the processor does not read from it directly.
3. The processor reads the instructions from memory **one after the other** and runs them.
4. When you close the program, its space in memory is freed for other programs.

1.2 A short history of computers

The computer did not appear in one day. These are the main steps:



Generation	Period	Main technology	Example
1st	1945–1955	vacuum tubes	ENIAC (30 tons!)
2nd	1955–1965	transistor	IBM 7090
3rd	1965–1980	integrated circuits	IBM 360
4th	1980–today	microprocessor (VLSI)	personal computer, phone

Each new generation brought a technology that made the parts **smaller**, **faster** and **less hungry for energy**:

- **Vacuum tubes** were big, got very hot, and burned out all the time. Some had to be changed almost every day.
- A **transistor** does the same job (an electric switch that opens and closes), but it is hundreds of times smaller and does not burn out.
- An **integrated circuit** puts thousands of transistors on one small piece of silicon, instead of connecting them with wires.
- A **microprocessor** puts the **whole** processor on one chip. This gave us the personal computer, then the smartphone.

Definition

Moore's law (Fr. *loi de Moore*): in 1965, Gordon Moore (a founder of Intel) noticed that the number of transistors on a chip **doubles about every two years**. This stayed true for more than fifty years: from 2 300 transistors in the 4004 to tens of billions today.

Did you know?

ENIAC filled a whole room and used 150 kilowatts. Your phone is millions of times more powerful and runs on a small battery. The reason: transistors became tiny, so one processor now holds billions of them.

1.3 Quick reminder: how does a computer count?

A computer only knows two electric states: **current flows** or **current does not flow**. We write them 1 and 0.

Definition

- A **bit** (Fr. *bit*): the smallest unit of information. Its value is 0 or 1.
- A **byte** (Fr. *octet*): a group of **8 bits**.
- A **word** (Fr. *mot*): the number of bits the processor handles at once (16 bits in the 8086).

We write numbers in three systems: decimal (base 10), binary (base 2), and hexadecimal (base 16, its digits are 0 to 9, then A to F).

Decimal	Binary	Hex	Decimal	Binary	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Step by step: convert $2F_H$ to binary and to decimal

1. Each hex digit is **4 bits**: $2 = 0010$ and $F = 1111$.
2. Put them side by side: $2F_H = 0010\,1111_2$.
3. To decimal: $2 \times 16 + 15 = 32 + 15 = 47$.

Step by step: convert the binary number 1011 to decimal

Each bit has a "weight", a power of 2. We start on the right with $2^0 = 1$:

Bit	1	0	1	1
Weight	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bit × weight	8	0	2	1

We add: $8 + 0 + 2 + 1 = 11$. So $1011_2 = 11$.

Step by step: convert the decimal number 13 to binary (divide by 2 again and again)

1. $13 \div 2 = 6$, remainder **1**.
2. $6 \div 2 = 3$, remainder **0**.
3. $3 \div 2 = 1$, remainder **1**.
4. $1 \div 2 = 0$, remainder **1**. We stop, because the quotient is 0.
5. Read the remainders **from the last to the first**: 1101_2. Check: $8 + 4 + 0 + 1 = 13$.

Everyday analogy

Why use hexadecimal? Because binary numbers are long and tiring to read: $1010\,1111\,0011\,0110_2$. Hexadecimal packs every 4 bits into one digit, so the number becomes $AF36_H$. It is the "short form" programmers use to write binary quickly, like writing "Dr." instead of "Doctor".

Watch out

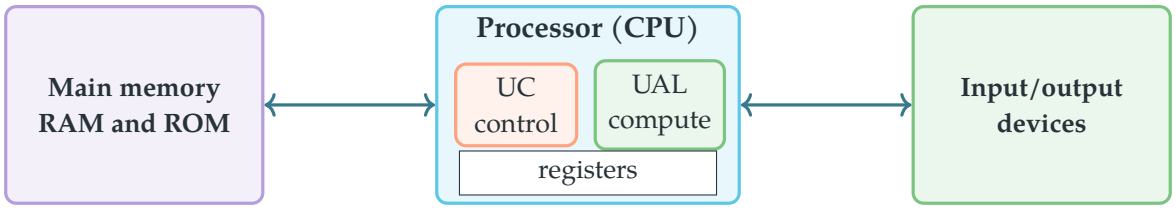
In assembly language we write `h` after a hex number (`2Fh`), and `b` after a binary number (`00101111b`). A number without a letter is decimal.

1.3.1 Memory size units

Unit	Symbol	Value
kilobyte	KB	$2^{10} = 1024$ bytes
megabyte	MB	$2^{20} = 1\,048\,576$ bytes
gigabyte	GB	2^{30} bytes (about one billion)
terabyte	TB	2^{40} bytes

1.4 The main parts of a computer

Inside every computer, from the biggest server to the smallest smartwatch, we find three main parts:



1. The **processor** (Fr. *processeur*) (CPU): the "brain" of the computer. It reads the instructions and runs them.
2. The **main memory** (Fr. *mémoire centrale*): it keeps the program that is running now, and its data.
3. The **input/output devices** (Fr. *périphériques d'E/S*): they let the computer talk with the outside world.

Step by step: the trip of a letter from the keyboard to the screen

You press the key **A** in a text editor:

1. The keyboard (**input**) sends the code of the key to the computer.
2. The processor is interrupted (it stops its work for a moment), reads the code, and stores it in **memory**, inside the text you are writing.
3. The text editor asks the processor to "draw" the letter: the processor computes the light dots that make the letter.
4. These dots go to the graphics card, then to the screen (**output**).

All this happens in a tiny part of a second, so the letter seems to appear at once.

Definition

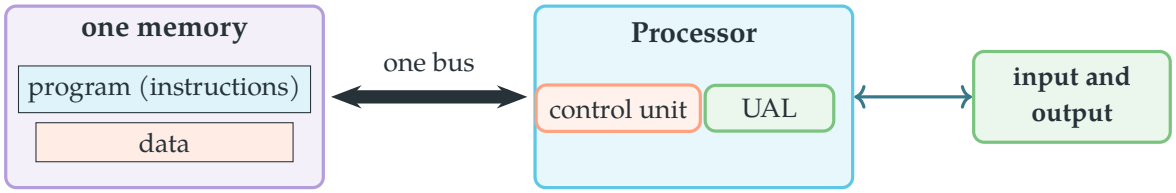
The **central unit** (Fr. *unité centrale*): the processor plus the main memory. This is the part that really runs the programs.

1.5 The two basic models: Von Neumann and Harvard

1.5.1 The Von Neumann model

In 1945, the scientist John von Neumann proposed a simple and brilliant idea: we keep the **program** and the **data** together in **the same memory**. Before that, changing the program meant re-connecting wires by hand!

In Von Neumann's model, a computer has **five units**: input, output, memory, the arithmetic and logic unit, and the control unit. The key idea is the **stored program** (Fr. *programme enregistré*): instructions are numbers, just like data, so we can keep them in the same memory, and read and change them the same way.



- **Advantage**: simple and flexible. To change the program, we just load another program into memory.
- **Drawback**: one bus for instructions and data, so we cannot read an instruction and a data item at the same moment. This traffic jam is called the **Von Neumann bottleneck** (Fr. *goulot de Von Neumann*).

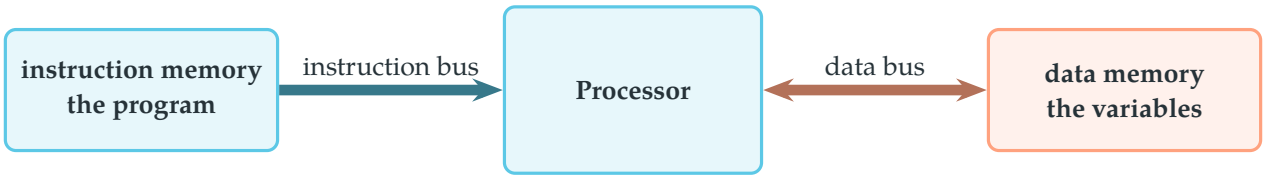
Everyday analogy

Imagine a library with **one narrow door**. The students who bring back books (the data) and the students who come for instructions (the program) all use the same door, one by one. Even if the

staff inside is very fast, the door decides the speed. This is the bottleneck.

1.5.2 The Harvard model

In this model we use **two separate memories**: one for instructions and one for data. Each one has its own bus.



- **Advantage**: faster, because the processor reads the next instruction and a data item at the same time.
- **Drawback**: more complex and more expensive, and the size of each memory is fixed.

Everyday analogy

In the Harvard library there are **two doors**: one for instructions and one for data. The two queues move at the same time, so people get in faster. But two doors cost more, and if one side is full we cannot use the free space of the other side.

1.5.3 Comparison and uses

	Von Neumann	Harvard
Memory	one memory for program and data	two separate memories
Buses	one shared bus	two independent buses
Speed	slower (bottleneck)	faster (two reads at once)
Complexity	simple and cheap	more complex, more expensive
Used in	personal computers, servers	microcontrollers (Arduino), signal processors (DSP)

Did you know?

The processors in your computer mix the two models. The main memory (RAM) is shared (Von Neumann), but inside the processor there are two small, fast, separate memories: one for instructions and one for data (Harvard). We call this **modified Harvard**. We come back to it in Chapter 5.

1.6 The processor: role and parts

Definition

The **processor** (Fr. *processeur, UCT*) (CPU): the circuit that runs the instructions of a program, one after the other.

The processor has three main parts (we study them in detail in Chapter 2):

Part	Its job	Analogy
control unit (UC)	reads the instruction, understands it, gives orders to the other parts	the orchestra conductor
arithmetic and logic unit (UAL)	does arithmetic ($+$, $-$) and logic (AND, OR)	the calculator
registers	very small and very fast memory inside the processor	the scrap paper on the desk

1.6.1 The clock and the speed of the processor

The processor works to the rhythm of a **clock** (Fr. *horloge*) that sends regular pulses. At each pulse the processor moves one small step forward in its work.

Definition

The **frequency** f: the number of pulses per second, measured in hertz (Hz). The **period** T: the length of one pulse, and $T = \frac{1}{f}$.

Step by step: a processor at 3 GHz

1. $f = 3\ \text{GHz} = 3 \times 10^9$ pulses per second.
2. $T = \frac{1}{3 \times 10^9} \approx 0.33 \times 10^{-9}$ second, so about 0.33 nanosecond.
3. In the time you blink (about 0.1 second), this processor does 300 million cycles!

Watch out

The frequency alone does not decide the speed of a processor. What matters is the **number of instructions done per second**. A 3 GHz processor that needs 4 cycles per instruction runs $\frac{3\times10^9}{4} = 750$ million instructions per second. A 2 GHz processor that runs 2 instructions per cycle runs 4×10^9 instructions per second: it is faster, even with a lower frequency.

1.7 Memory

1.7.1 What does memory look like inside?

Imagine memory as a **big building full of boxes**. Each box has a number called the **address**, and each box holds **one byte**.

address	content
0000	3A
0001	FF
0002	07
0003	41
0004	00

Address 0003 holds 41_{H}.
The address never changes; the content can change.

Everyday analogy

The address is like the **house number** in a street, and the content is like **the person who lives there**. The house number is fixed; the people living there can change.

1.7.2 Only two operations: read and write

- **Read**: the processor sends an address, and the memory gives back the content (the content does not change).
- **Write**: the processor sends an address and a value, and the memory stores the value instead of the old one.

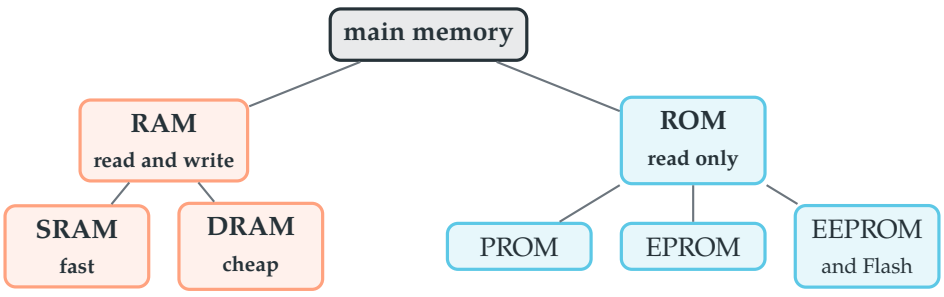
Example

Box 0002 holds 07_{H}. The processor runs "write 55_{H} at 0002". After that, the box holds 55_{H}, and 07_{H} is lost for ever. So we must be careful where we write: writing in the wrong place erases important data.

Definition

Random Access Memory (Fr. *mémoire à accès aléatoire*) (RAM): "random access" means we can reach **any box directly**, in the same time, without going through the boxes before it. This is the opposite of an old cassette tape, which you must wind until you reach the song.

1.7.3 Types of memory



Type	Description	When power is off	Used for
SRAM	transistors only, very fast and expensive	erased	cache memory inside the processor
DRAM	a tiny capacitor per bit, must be "refreshed" all the time	erased	main memory (RAM sticks)
ROM	written once in the factory	kept	old boot programs
EPROM	erased with ultraviolet light, then written again	kept	prototypes
EEPROM / Flash	erased electrically	kept	BIOS/UEFI, USB keys, SSD

Everyday analogy

SRAM is like a light switch: it stays in its position (on or off) as long as there is power. **DRAM** is like a glass with a small hole: the water (the electric charge) leaks slowly, so we must fill it again thousands of times per second. This is called **refresh**. But the glass is much smaller than the switch, so we can put billions of them on a cheap memory stick.

Did you know?

Why does a computer need ROM or Flash? When you switch it on, the RAM is completely empty. The processor needs a first program to start with: the boot program (BIOS or UEFI), kept in a memory that is not erased. This program checks the hardware, then finds the operating system on the disk and loads it into RAM.

Definition

A **volatile** (Fr. *volatile*) memory loses its content when the power is off (RAM). A **non-volatile** (Fr. *non volatile*) memory keeps it (ROM, Flash).

1.7.4 Calculating the size of a memory

If the processor has n address lines (wires), it can write 2^n different addresses.

Definition

number of boxes $= 2^n$ and size $= 2^n \times$ size of one box

Step by step: the memory size of the 8086

The 8086 has a **20-bit** address bus, and each box holds one byte.

1. Number of addresses: 2^{20}.
2. $2^{20} = 2^{10} \times 2^{10} = 1024 \times 1024 = 1\,048\,576$ addresses.
3. Size $= 2^{20} \times 1$ byte $= 1$ megabyte (1 MB).

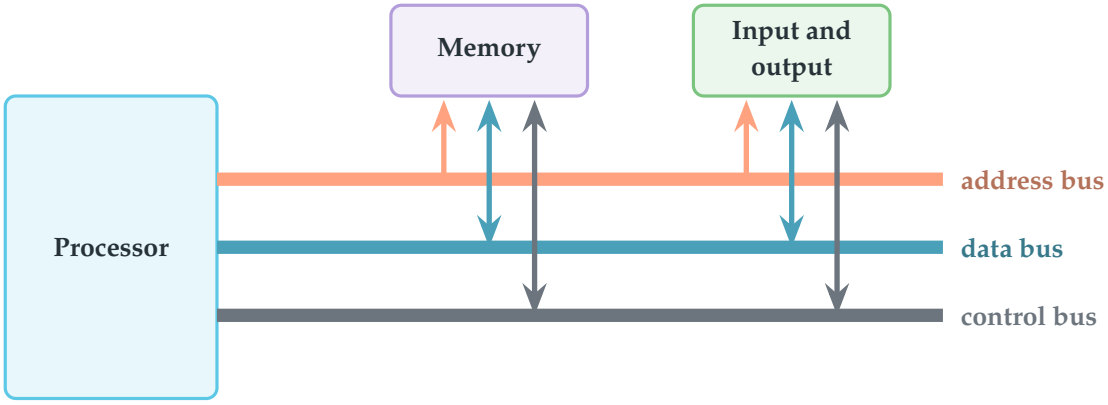
Step by step: how many address lines for a 64 KB memory?

1. $64\ \text{KB} = 64 \times 1024 = 2^6 \times 2^{10} = 2^{16}$ bytes.
2. So $2^n = 2^{16}$, and $n = 16$ lines.

1.8 Buses

Definition

A **bus** (Fr. *bus*): a group of parallel wires that carries information between the processor, the memory and the input/output devices. Each wire carries one bit at a time.



Bus	Direction	What does it carry?
address bus	from the processor only	the number of the box the processor wants to read or write
data bus	both ways	the value that is read or written
control bus	both ways	signals such as read (RD), write (WR), clock, interrupt

Everyday analogy

Imagine you ask for a parcel at the post office. The **address** tells the clerk which shelf you mean (address bus). The **parcel** itself is the data (data bus). The words "give me" or "take this" are the control signal (read or write).

Step by step: the processor reads box 0003

1. The processor puts the address 0003 on the **address bus**.
2. It sends a **read** signal (RD) on the **control bus**.
3. The memory puts the content of the box (41_H) on the **data bus**.
4. The processor takes the value 41_H and keeps it in one of its registers.

1.8.1 Bus width

- The **width of the address bus** decides the **maximum memory size**: n lines give 2^n addresses.
- The **width of the data bus** decides **how many bits move at once**: a 16-bit bus moves two bytes at a time.

Step by step: moving 1000 bytes with different buses

1. An **8-bit** data bus moves one byte at a time: we need 1000 transfers.
2. A **16-bit** bus moves two bytes: $1000 \div 2 = 500$ transfers.
3. A **64-bit** bus moves 8 bytes: $1000 \div 8 = 125$ transfers only.

The wider the bus, the faster the data moves, just as more cars pass on an 8-lane motorway than on a one-lane road.

Processor	Data bus	Address bus	Max memory
Intel 8086	16 bits	20 bits	$2^{20} = 1$ MB
Intel 80386	32 bits	32 bits	$2^{32} = 4$ GB
modern x86-64	64 bits	48 bits (in practice)	$2^{48} = 256$ TB

1.9 Exercises

Exercise 1.1

Convert these numbers: (a) $3C_H$ to decimal and binary. (b) The decimal number 200 to hexadecimal.

Solution

(a) $3C_H = 3 \times 16 + 12 = 60$, and in binary $0011\,1100_2$.
(b) $200 = 12 \times 16 + 8$, and 12 is written C, so $200 = C8_H$.

Exercise 1.2

A processor has a 24-bit address bus, and each memory box holds one byte. What is the maximum memory size?

Solution

$2^{24} = 2^4 \times 2^{20} = 16 \times 1$ MB $= 16$ MB.

Exercise 1.3

We want a 4 GB memory. How many lines does the address bus need?

Solution

$4\,\text{GB} = 2^2 \times 2^{30} = 2^{32}$ bytes, so we need **32 lines**.

Exercise 1.4

True or false? Explain. (a) In the Von Neumann model, the program is kept in a different memory from the data. (b) The address bus carries information in both directions. (c) DRAM keeps its content after the computer is switched off.

Solution

(a) **False**: Von Neumann has one memory for program and data; separation is the Harvard model.
(b) **False**: only the processor sends addresses, so it goes one way.
(c) **False**: DRAM is volatile; it is erased when the power goes off.

Exercise 1.5

A processor runs at 2 GHz, and each instruction needs 4 cycles on average. How many instructions does it run per second?

Solution

Cycles per second $= 2 \times 10^9$. Instructions $= \dfrac{2 \times 10^9}{4} = 0.5 \times 10^9$, that is **500 million instructions per second**.

Chapter 2

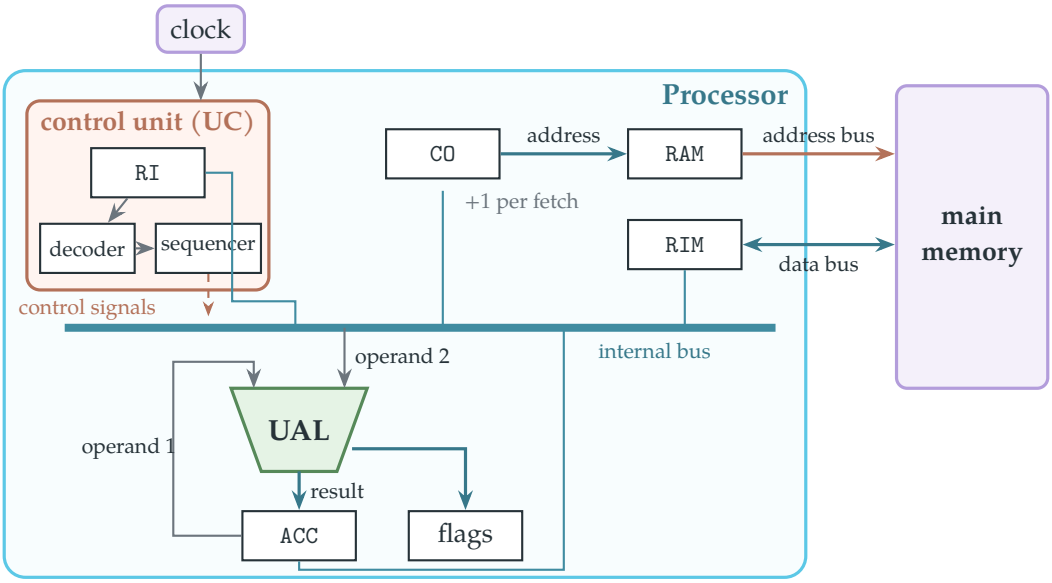
Inside the Processor

Summary: what will we learn in this chapter?

- What is inside the processor, and the job of each part.
- Registers and their types.
- The arithmetic and logic unit: how the processor adds two numbers with logic gates, and what flags are.
- The control unit: who decides what happens at each moment.
- The instruction set and the addressing modes.
- The steps of every instruction: fetch, decode, execute, write back.

2.1 A look inside the processor

Let us open the processor and look inside. The figure shows a simple processor, but it has all the main ideas of real processors.



Part	Its job in one sentence
CO (program counter)	keeps the address of the **next instruction**
RI (instruction register)	keeps the **current instruction** being run
RAM (memory address register)	keeps the address we send to memory
RIM (memory data register)	keeps the data coming from memory or going to it
ACC (accumulator)	keeps the result of the calculation
flags	keep information about the last result: is it zero? negative? ...
UAL	does the arithmetic and logic operations
decoder and sequencer	understand the instruction and send the orders in the right order

Watch out

Do not mix up RAM the memory (*Random Access Memory*) and RAM the register (*Registre d'Adresse Mémoire*) used in French books. In English this register is called **MAR**, RIM is called **MDR**, and CO is called **PC** (*Program Counter*). In the 8086 it is called **IP**. In this course we keep the French short names (CO, RI, RAM, RIM, UAL, UC) because they are used in the syllabus.

2.2 Registers

Definition

A **register** (Fr. *registre*): a very small memory box (8, 16, 32 or 64 bits) **inside the processor**. It is the fastest memory in the whole computer.

Everyday analogy

When you solve a maths exercise, you do not walk to the library each time you need a number! You write the numbers you need on **scrap paper** in front of you. Registers are the processor's scrap paper, and memory is the library.

2.2.1 Why are registers important for speed?

Place	Access time (about)	If one cycle took one second...
register	less than 1 ns	one second
main memory (RAM)	about 80 ns	about a minute and a half
SSD disk	about 100 µs	more than a day

So we keep in registers the data we use often, and we go to memory as little as possible.

2.2.2 Types of registers

1. **Data registers** (Fr. *registres de données*): they keep the numbers we compute with (like AX, BX in the 8086).
2. **Address registers** (Fr. *registres d'adresse*): they keep memory addresses (like SI, DI, SP).
3. **Control and status registers** (Fr. *registres de contrôle*): they help the processor organise its work: CO, RI, and the flags register.

Example: registers at work

The processor is now running the instruction at address 11, "add ACC and the content of box 21".

- CO holds 12: the address of the **next** instruction, not the current one.
- RI holds the current instruction, so the control unit can read it during the whole execution.
- RAM holds 21: the address we are sending to memory now.
- RIM receives the content of box 21 coming from memory.
- ACC holds the first number, and will receive the result.

Each register has a clear job. This is why we call them special registers, unlike the general registers that the programmer uses freely.

2.2.3 The flags register

Definition

A **flag** (Fr. *indicateur, drapeau*): one bit that the arithmetic unit sets to 1 or 0 after each operation, depending on the result. We use flags later to make decisions (for example: "if the result is zero, jump").

Flag	Becomes 1 when...	Example on 8 bits
ZF (zero)	the result is **zero**	$5 - 5 = 0$
CF (carry)	a carry comes out of the last bit (the result is too big)	$200 + 100 = 300 > 255$
SF (sign)	the last bit of the result is 1 (negative number)	$3 - 7 = -4$
OF (overflow)	the result is wrong if we read the numbers as signed	$100 + 50 = 150 > 127$

2.2.4 The difference between CF and OF

The same byte can be read in two ways, and the processor does not know which one the programmer means:

- **unsigned**: values from 0 to 255.
- **signed**: values from -128 to $+127$. The last bit (bit 7) is the sign: 1 means negative.

So the processor computes both flags. CF matters for unsigned numbers, and OF matters for signed numbers.

Step by step: $100 + 50$ on 8 bits

1. $100 = 0110\,0100_2$ and $50 = 0011\,0010_2$.
2. The sum: $1001\,0110_2 = 150$ (unsigned). No carry out of bit 7, so CF = 0: correct for unsigned numbers.
3. But bit 7 is now 1! Reading it as signed, we see a negative number: $150 - 256 = -106$.
4. We added two positive numbers and got a negative one: impossible, so the signed result is wrong, and OF = 1.

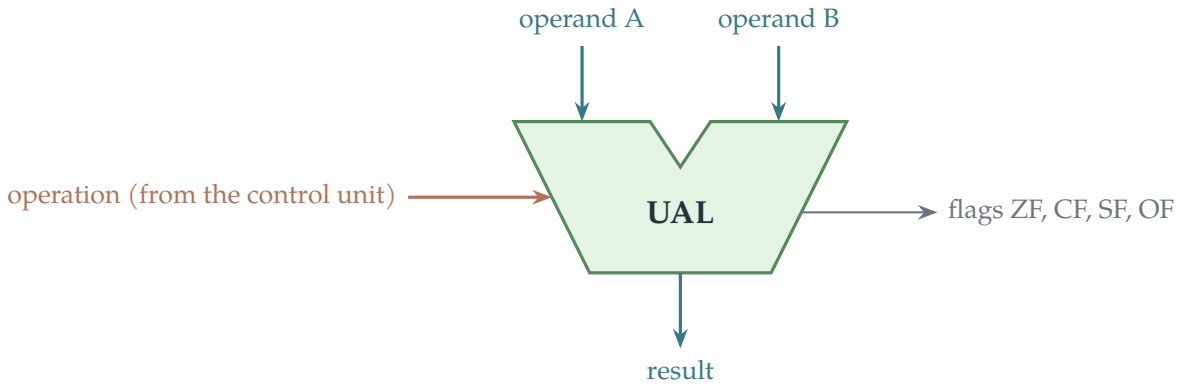
Watch out

An easy rule for OF: overflow happens when we add two numbers **with the same sign** and get a result **with the other sign**. Adding a positive and a negative number never overflows.

2.3 The arithmetic and logic unit (UAL)

Definition

The **arithmetic and logic unit** (Fr. *unité arithmétique et logique, UAL*) (ALU in English): the part that does arithmetic (add, subtract, compare...) and logic (AND, OR, XOR, NOT, shifts).



2.3.1 How do logic gates add? The half adder

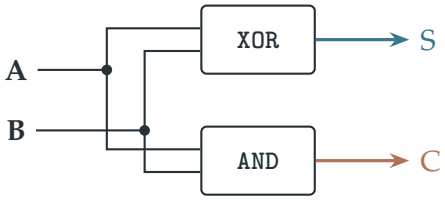
Let us add two bits A and B. There are only four cases:

A	B	sum S	carry C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	**1**

We see that S is 1 when exactly one bit is 1: this is the **XOR** gate. And C is 1 when both bits are 1: this is the **AND** gate.

Definition

Half adder (Fr. *demi-additionneur*): $S = A \oplus B$ and $C = A \cdot B$



2.3.2 The full adder

To add numbers with many bits, we must also add the **incoming carry** from the previous bit. A full adder has three inputs: A, B and the carry in C_{in}.

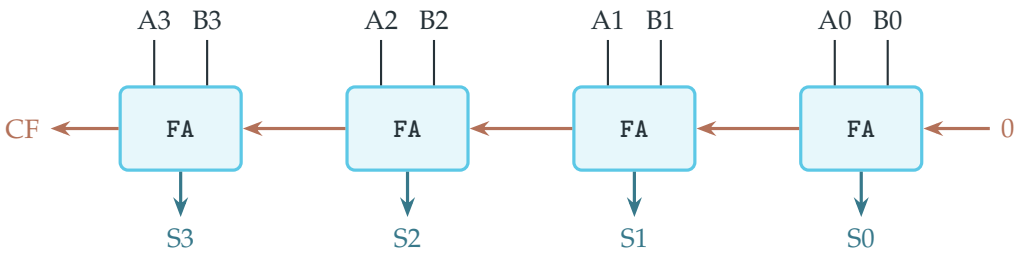
Definition

Full adder (Fr. *additionneur complet*): $S = A \oplus B \oplus C_{in}$ and $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.3.3 A 4-bit adder

We put 4 full adders side by side: the carry out of each one goes into the next one, exactly as we do when we add with pen and paper.



Bit 0 (the smallest) is on the right, like when we write numbers. The carry travels from right to left.

Step by step: add $0111 + 0110$ (that is $7 + 6$) with the adder

Bit	A	B	C_{in}	S	C_{out}
0	1	0	0	1	0
1	1	1	0	0	1
2	1	1	1	1	1
3	0	0	1	1	0

Read S from bit 3 down to bit 0: $1101 = 13$, which is indeed $7 + 6$. The last carry is 0, so CF = 0.

2.3.4 And subtraction? The same adder!

The processor does not need a special circuit to subtract. It uses the **two's complement** (Fr. *complément à deux*): $A - B = A + (-B)$, and we get $-B$ by flipping all the bits of B, then adding 1.

Step by step: $7 - 5$ on 4 bits

1. $5 = 0101$. Flip the bits: 1010. Add 1: 1011. So $-5 = 1011$.
2. Add: $0111 + 1011 = 1\,0010$. We drop the carry out, so $0010 = 2$ is left.
3. The result: $7 - 5 = 2$. Correct! The same circuit added and subtracted.

2.3.5 Comparing by subtracting

The processor does not need a special circuit to compare either: to compare A and B it computes $A - B$ **without keeping the result**, then looks at the flags:

- if ZF = 1 then $A = B$.
- if CF = 1 (a "borrow" happened) then $A < B$ (for unsigned numbers).

2.3.6 A real test: how do the flags change?

We ran the program below on the 8086. Do not worry if you do not understand all the instructions yet; we study them in Chapter 3. Just look at the flags.

```
1  org 100h
2
3      mov al, 200       ; AL = 200
4      add al, 100       ; 300 does not fit in 8 bits -> carry
5      mov al, 100       ; AL = 100
6      add al, 50        ; 150: too big for a signed byte -> overflow
7      mov al, 5
8      sub al, 5         ; result 0 -> zero flag
9      mov al, 3
10     sub al, 7         ; negative result -> sign flag and borrow
11     ret
```

Line	Instruction	AL	CF	ZF	SF	OF
4	`add al, 100` $(200+100)$	$2C_H = 44$	1	0	0	0
5	`mov al, 100`	$64_H = 100$	1	0	0	0
6	`add al, 50` $(100+50)$	$96_H = 150$	0	0	1	1
8	`sub al, 5` $(5-5)$	$00_H = 0$	0	1	0	0
10	`sub al, 7` $(3-7)$	$FC_H = 252$	1	0	1	0

Step by step: why did $200+100$ give 44?

1. $200 + 100 = 300$, but AL holds only 8 bits, that is from 0 to 255.
2. $300 - 256 = 44$: AL keeps 44, and the "256" that did not fit goes into the flag CF, so CF = 1.

Watch out

Look at line 5: the instruction `mov` **does not change the flags**, so CF stayed 1. Only arithmetic and logic instructions change the flags.

2.4 The control unit (UC)

Definition

The **control unit** (Fr. *unité de commande, UC*): the part that reads the instructions and understands them, then sends **signals** to the other parts at each clock pulse (open this register, add, read memory...) in the right order.

Everyday analogy

The control unit is the **orchestra conductor**: it does not play any instrument itself, but it tells each musician when to start and when to stop. Without it, everybody plays at the same time and it becomes noise.

Its main parts:

- the **instruction register** (RI): keeps the current instruction.

- the **decoder** (Fr. *décodeur*): reads the operation code (opcode) and finds what is asked: add? move? jump?
- the **sequencer** (Fr. *séquenceur*): sends the right signals, one after the other, to the rhythm of the clock.

2.4.1 Micro-operations, pulse by pulse

Each step of an instruction is split into very small steps called **micro-operations** (Fr. *micro-opérations*). At each clock pulse, the sequencer sends signals that open "doors" between registers. Here is how an instruction is fetched:

Pulse	Micro-operation	Signal sent by the sequencer
T1	RAM ← CO	"copy CO into RAM"
T2	RIM ← [RAM], CO ← CO + 1	"read memory" and "increase CO"
T3	RI ← RIM	"copy RIM into RI"
T4...	depends on the instruction	the decoder chooses the next signals

The fetch is the same for all instructions; what comes after depends on the opcode.

2.4.2 Two kinds of control units

	Hardwired (câblée)	Microprogrammed (microprogrammée)
Idea	a fixed logic circuit makes the signals	each instruction is a small program stored in an internal memory
Speed	very fast	a little slower
Changes	hard (we must change the circuit)	easy (we change the microprogram)
Found in	RISC processors	CISC processors like the 8086

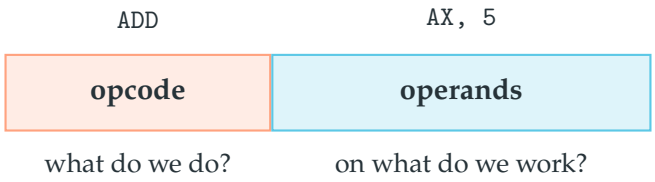
2.5 The instruction set

Definition

The **instruction set** (Fr. *jeu d'instructions*): the list of all the instructions a processor can run. Each processor family has its own set.

2.5.1 The shape of an instruction

Every machine-language instruction has two parts:



Definition

Assembly language (Fr. *langage d'assemblage*): we write instructions with short words that humans understand (`ADD`, `MOV`) instead of binary numbers. A program called the **assembler** (Fr. *assembleur*) turns them into machine language (zeros and ones).

Example

On the 8086, the instruction `add al, 5` is written in machine language with the two bytes 04 05, that is 0000 0100 0000 0101: the first byte is the opcode "add an immediate value to AL", and the second one is the number 5. The programmer writes `add al, 5`, and the assembler writes the zeros and ones (we see this in detail in Chapter 3).

2.5.2 How many instructions can a processor have?

If the opcode uses k bits, there are 2^k different codes, so at most 2^k instructions.

Step by step

A 6-bit opcode: $2^6 = 64$ possible instructions. An 8-bit opcode: $2^8 = 256$ instructions.

2.5.3 Number of addresses in an instruction

Let us compute $A = B + C$ on four kinds of machines. The fewer operands in an instruction, the shorter the instruction, but the more instructions we need:

Kind of machine	Program	Instructions
3 addresses	`ADD A, B, C`	1
2 addresses	`MOV A, B` then `ADD A, C`	2
1 address (accumulator)	`LOAD B` then `ADD C` then `STORE A`	3
0 address (stack)	`PUSH B`, `PUSH C`, `ADD`, `POP A`	4

The 8086 is a **2-address** machine: `add ax, bx` means AX $\leftarrow$ AX + BX.

2.5.4 Classes of instructions

Class	Job	Examples (8086)
data transfer	copy a value from one place to another	`MOV`, `PUSH`, `POP`
arithmetic	add, subtract, multiply, divide	`ADD`, `SUB`, `MUL`, `INC`
logic	operations on bits	`AND`, `OR`, `XOR`, `SHL`
flow control	jump and change the order of execution	`JMP`, `JE`, `CALL`, `RET`
input/output	talk with devices	`IN`, `OUT`, `INT`

2.5.5 CISC and RISC

- **CISC** (Complex Instruction Set Computer): many complex instructions; some do a lot of work. Example: Intel x86.
- **RISC** (Reduced Instruction Set Computer): few, simple, fast instructions, all of the same length. Example: ARM in phones, RISC-V.

2.6 Addressing modes

Definition

An **addressing mode** (Fr. *mode d'adressage*): the way we say **where the data is** that the instruction works on.

Everyday analogy

You want to give a book to a friend: **(1)** you put the book in his hand (immediate), **(2)** you say: "the book is on shelf number 12" (direct), **(3)** you say: "look at the paper in my pocket, it has the shelf number" (indirect), **(4)** you say: "the third shelf after shelf number 10" (indexed).

2.6.1 The basic modes

Mode	8086 example	Where is the data?
immediate	`mov ax, 5`	inside the instruction itself
register	`mov bx, ax`	in a register
direct	`mov cx, [val]`	in memory; its address is written in the instruction
indirect	`mov dx, [si]`	in memory; its address is in a register
indexed	`mov ax, [tab+di]`	address = start address + value of an index register
relative	`jmp next`	address = current address + offset (for jumps)

2.6.2 When do we use each mode?

- **Immediate**: for constants (number of loop turns, start value). The fastest mode, because the value came with the instruction, so we do not go to memory. But we cannot change it while the program runs.
- **Register**: for fast calculations between values already inside the processor. Very fast too.
- **Direct**: for variables that have a fixed place in memory (like a global variable). One memory read.
- **Indirect**: when we do not know the address when we write the program, but compute it while it runs. It is the basis of **pointers** in C.
- **Indexed**: for arrays. We fix the start address and change the index at each turn of the loop, to visit every element.
- **Relative**: for jumps. Because the distance is relative, the program works even if it is loaded at another place in memory.

Step by step: how many times do we go to memory?

Instruction	Memory reads to get the data
`mov ax, 5` (immediate)	0
`mov ax, bx` (register)	0
`mov ax, [val]` (direct)	1
`mov ax, [si]` (indirect with a register)	1
memory indirect (the address itself is in memory)	2

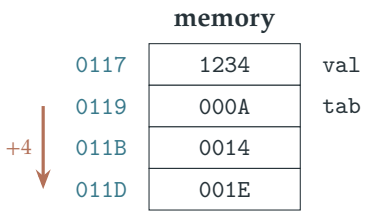
Every trip to memory costs time, so programmers try to keep important data in registers.

2.6.3 The modes in a real program

```
org 100h              ; .COM program starts at offset 100h

    mov ax, 5         ; immediate : the value 5 is inside the instruction
    mov bx, ax        ; register  : copy register AX into BX
    mov cx, [val]     ; direct    : read memory at the address of val
    lea si, val       ; SI = address of val
    mov dx, [si]      ; indirect  : read memory at the address in SI
    mov di, 4         ; DI = 4 (two words further)
    mov ax, [tab+di]  ; indexed   : read tab + 4 = third element
    ret               ; end of program

val dw 1234h          ; a word in memory
tab dw 10, 20, 30     ; an array of 3 words
```



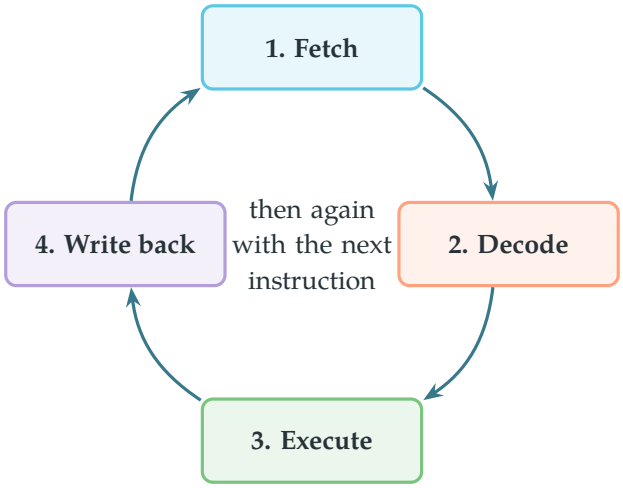
Line	What does it do?
1	A program of type .COM: it starts at address 100_H.
3	**Immediate**: the number 5 is written inside the instruction. Result: AX = 0005.
4	**Register**: copy AX into BX. Result: BX = 0005.
5	**Direct**: read memory at the address of `val` (0117_H). Result: CX = 1234.
6	`lea` puts the **address** of `val` (not its value) into SI: SI = 0117.
7	**Indirect**: read memory at the address held in SI. Result: DX = 1234.
8	DI = 4: each element of `tab` is a word (two bytes), so the third element is 4 bytes away.
9	**Indexed**: address = $0119_H + 4 = 011D_H$, which holds 30. Result: AX = 001E.
10	End of the program, back to the system.

Watch out

The brackets `[ ]` always mean: "the **content of memory** at this address". `mov dx, si` copies the number 0117_H itself, but `mov dx, [si]` reads what is stored at address 0117_H.

2.7 The steps of an instruction

Every instruction, even a simple one, goes through the same steps. This is called the **instruction cycle** (Fr. *cycle d'instruction*).



1. **Fetch**: CO is sent to RAM, memory is read, the instruction arrives in RIM, then in RI. Then CO increases to point to the next instruction.
2. **Decode**: the decoder reads the opcode in RI and finds the operands.
3. **Execute**: if we need data from memory we read it, then the UAL computes the result.

4. **Write back**: we store the result in a register or in memory, and we update the flags.

2.7.1 A complete example, step by step

We take a simple machine with an accumulator ACC, and we want to compute $z = x + y$. The variable x is at address 20, y at 21, and z at 22.

Address	Content	Meaning
10	`LOAD 20`	ACC ← [20]
11	`ADD 21`	ACC ← ACC + [21]
12	`STORE 22`	[22] ← ACC
20	7	x
21	5	y
22	0	z (before running)

We follow the second instruction `ADD 21` step by step (at the start, CO = 11 and ACC = 7):

Step	What happens?	CO	RAM	RIM	RI	ACC
fetch	RAM ← CO	11	11	–	–	7
	read memory: RIM ← [11]	11	11	`ADD 21`	–	7
	RI ← RIM, and CO ← CO + 1	12	11	`ADD 21`	`ADD 21`	7
decode	the decoder understands: "add the content of address 21"	12	11	`ADD 21`	`ADD 21`	7
execute	RAM ← 21, then read: RIM ← [21]	12	21	5	`ADD 21`	7
	UAL: 7 + 5	12	21	5	`ADD 21`	7
write back	ACC ← 12, and update the flags	12	21	5	`ADD 21`	**12**

Summary

After this instruction: ACC = 12 and CO = 12, so the next instruction is `STORE 22`, which writes 12 in memory, and z becomes 12.

Did you know?

Modern processors do not wait for one instruction to finish before starting the next. While one instruction runs, the next one is decoded and the third one is fetched. This is the **pipeline**, which we study in Chapter 4.

2.8 Exercises

Exercise 2.1

Compute with the full adder (bit by bit) $1011 + 0110$ on 4 bits. What is the result, and the value of CF?

Solution

Bit	A	B	C_{in}	S	C_{out}
0	1	0	0	1	0
1	1	1	0	0	1
2	0	1	1	0	1
3	1	0	1	0	1

The result is 0001 and CF = 1: $11 + 6 = 17 = 16 + 1$, and the "16" went into CF.

Exercise 2.2

On 8 bits, give AL, CF and ZF after: (a) $255 + 1$ (b) $10 - 10$ (c) $10 - 20$.

Solution

(a) $256 - 256 = 0$: AL = 0, CF = 1, ZF = 1.
(b) AL = 0, CF = 0, ZF = 1.
(c) $10 - 20 = -10$, and on 8 bits $256 - 10 = 246 = \mathrm{F6_H}$: CF = 1 (borrow), ZF = 0.

Exercise 2.3

Give the addressing mode of each instruction: (a) `mov al, [bx]` (b) `mov al, 10h` (c) `mov al, [10h]` (d) `mov al, bl`.

Solution

(a) register indirect (b) immediate (c) direct (d) register.

Exercise 2.4

In the example above ($z = x + y$), write the table of steps for the third instruction `STORE 22`.

Solution

Fetch: RAM ← 12, RIM ← STORE 22, RI ← RIM, CO = 13. **Decode**: "write ACC at address 22". **Execute and write**: RAM ← 22, RIM ← ACC (12), then a write signal: $[22] = 12$.

Chapter 3

Case Study: the Intel 8086 Processor

Summary: what will we learn in this chapter?

- The 80x86 family, and the internal structure of the 8086.
- The 8086 registers, and segmented addressing (how to compute a physical address).
- Writing a first program, and how an instruction runs inside the processor.
- The instructions: transfer, arithmetic, logic, jumps, loops, then how an instruction becomes machine code.
- Input/output, the stack, procedures, and interrupts.

3.1 The 80x86 family

In 1978 Intel released the 8086. It was a big success because IBM chose its brother, the 8088, for the first personal computer (IBM PC) in 1981. Since then, every new Intel and AMD processor can still run 8086 programs. This family is called **x86**.

Definition

Backward compatibility (Fr. *compatibilité ascendante*): a new processor understands all the instructions of the older processors of its family. A program written for the 8086 in 1980 can run on a modern Core i7. This is why the x86 family has lived for more than 45 years.

Processor	Year	Bits	Max memory	What is new
8086 / 8088	1978	16	1 MB	first processor of the family
80286	1982	16	16 MB	protected mode
80386	1985	32	4 GB	first 32-bit processor (IA-32)
80486	1989	32	4 GB	cache memory and floating-point unit inside
Pentium	1993	32	4 GB	runs two instructions at once
Athlon 64 (AMD)	2003	64	2^{48} in practice	the AMD64 (x86-64) architecture
Core i7 and later	2008	64	huge	several cores on the same chip

Definition

General features of the 8086:

- a **16-bit** processor: its registers and its data bus are 16 bits wide.

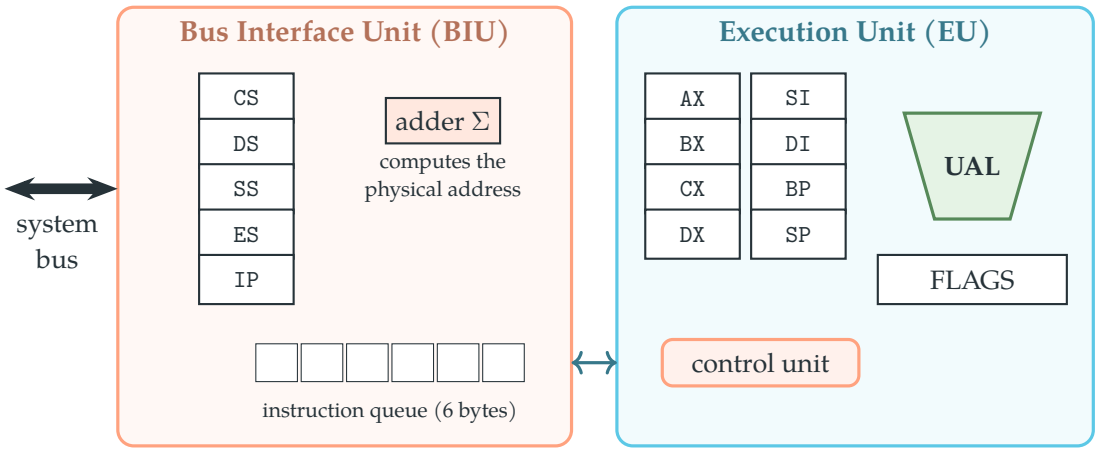
- a **20-bit** address bus: it can address 1 MB of memory.
- about 29 000 transistors, and a frequency between 5 and 10 MHz.
- about **100 different instructions** (a CISC processor).

Did you know?

The only difference between the 8086 and the 8088 is the **external data bus**: 16 bits in the 8086, only 8 bits in the 8088. Inside, they are identical. IBM chose the 8088 because 8-bit support chips were cheaper at that time.

3.2 Inside the 8086

The 8086 is split into two units that work **at the same time**:



- The **Bus Interface Unit** (Fr. *unité d'interface de bus*) (BIU): it talks with memory. It computes the addresses, fetches instructions in advance into a **queue** of 6 bytes, and reads and writes data.
- The **Execution Unit** (Fr. *unité d'exécution*) (EU): it takes the instructions from the queue, decodes them, and runs them with the arithmetic and logic unit.

Everyday analogy

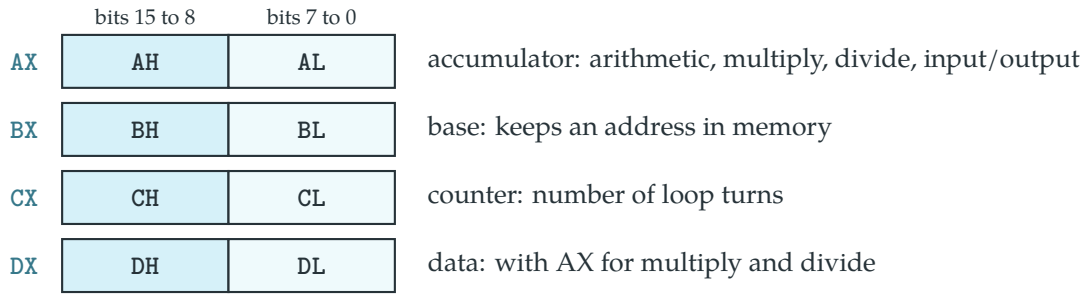
In a restaurant, the **waiter** (BIU) brings orders from the customers and puts them in a queue in front of the cook, while the **cook** (EU) prepares the current order. The cook never waits for the waiter, because the next order is always in the queue. This is the first idea of a pipeline!

3.3 The 8086 registers

The 8086 has **14 registers** of 16 bits.

3.3.1 General registers

Four data registers. Each one can be split into two halves of 8 bits:



Example

If AX = 1234_H then AH = 12_H and AL = 34_H. If we change AL to FF_H, AX becomes $12FF_H$.

Why split the registers? Because we often work with small values (a character, a number from 0 to 255). Instead of wasting a whole 16-bit register for one character, we can put two characters in one register: one in AH and one in AL.

Watch out

These registers are "general" because we can use them in any calculation, but some instructions force a register: `loop` always uses CX, `mul` and `div` always use AX and DX, and the `int 21h` services expect the function number in AH. You must remember these roles.

3.3.2 Pointer and index registers

Register	Name	Job
SI	Source Index	index of the source (arrays and strings)
DI	Destination Index	index of the destination
SP	Stack Pointer	points to the top of the stack
BP	Base Pointer	to reach data inside the stack

Example

To walk through an array of 10 elements, we put its start address in SI, then increase SI at each turn (`inc si`). At the first turn SI points to the first element, at the second turn to the second one, and so on. SI plays the role of the **finger** you put on the element you are reading now.

3.3.3 Segment registers

Register	Name	Points to the start of...
CS	Code Segment	the program segment (the instructions)
DS	Data Segment	the data segment (the variables)
SS	Stack Segment	the stack segment
ES	Extra Segment	an extra segment (for copying strings, for example)

3.3.4 Instruction pointer and flags

- **IP** (Instruction Pointer): the address of the next instruction inside the code segment. It is the CO we saw in Chapter 2. We cannot change it directly with `mov`, only with jumps and calls.
- **FLAGS**: 9 bits are used out of 16:



Flag	Meaning	Flag	Meaning
CF	carry or borrow out of the last bit	OF	overflow: wrong result for signed numbers
PF	the number of 1s in the low byte is even	DF	direction for strings (0: forward)
AF	carry from bit 3 to bit 4	IF	hardware interrupts allowed
ZF	the result is zero	TF	step-by-step execution (for debugging)
SF	the result is negative (last bit is 1)		

3.4 Segmented addressing

3.4.1 The problem

The 8086 registers are 16 bits wide, so they can write addresses from 0 to $2^{16} - 1 = 65535$ only (that is 64 KB). But the address bus is 20 bits wide (1 MB). How can we write a 20-bit address with 16-bit registers?

3.4.2 The solution: segment and offset

We use **two registers**: a **segment** register gives the start of a zone of memory, and the **offset** gives the distance inside this zone. We write the logical address as `segment:offset`.

Definition

$$\text{physical address} = \text{segment} \times 16 + \text{offset}$$

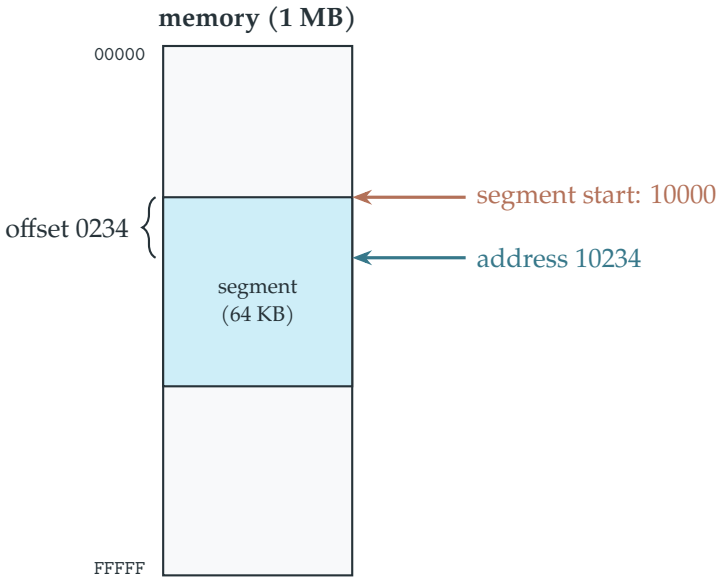
Multiplying by 16 = adding one zero on the right in hexadecimal.

Step by step: the physical address for CS = 1000H and IP = 0234H

1. Multiply the segment by 16 (add a zero): 1000_{H} becomes 10000_{H}.
2. Add the offset:

```
  10000
+  0234
-------
  10234
```

3. The physical address is 10234_{H}, a 20-bit address (5 hex digits).



3.4.3 Why multiply by 16?

If we used the segment as it is (without multiplying), we could not go beyond $\text{FFFF}_\text{H} + \text{FFFF}_\text{H} = 1\text{FFFE}_\text{H}$, about 128 KB only. With the multiplication by 16, a segment can point anywhere in 1 MB: $\text{FFFF}_\text{H} \times 16 = \text{FFFF0}_\text{H}$. So each segment always starts at an address that can be divided by 16. This block of 16 bytes is called a **paragraph** (Fr. *paragraphe*).

Step by step: which zone does the segment DS = 2000H cover?

1. The smallest offset is 0000_H: $20000_\text{H} + 0000_\text{H} = 20000_\text{H}$.
2. The biggest offset is FFFF_H: $20000_\text{H} + \text{FFFF}_\text{H} = 2\text{FFFF}_\text{H}$.
3. The segment covers the zone from 20000_H to 2FFFF_H, that is $2^{16} = 65536$ bytes = 64 KB.

Everyday analogy

Imagine a town with many streets. The full address of a house is "**street** number 12, **house** number 34". The segment register is the street number (it gives the start of a zone), and the offset is the house number inside the street. The postman needs both pieces of information to find the house.

Watch out

One physical address can be written in several ways: 1000:0234 and 1023:0004 give the same address 10234_H. Check it yourself: $10230_\text{H} + 0004_\text{H} = 10234_\text{H}$.

3.4.4 Default pairs

Each offset register is used by default with a given segment register:

Offset	Default segment	Use
IP	CS	fetching instructions
SP, BP	SS	the stack
BX, SI, direct address	DS	variables
DI (in string instructions)	ES	destination of a copy

Did you know?

In the .COM programs we write in this chapter, the four segments CS = DS = SS = ES are equal: the whole program fits in one 64 KB segment, and starts at offset 100_H. This is the meaning of the line `org 100h` that we find in all our programs.

3.5 8086 addressing modes

Definition

The **effective address** (Fr. *adresse effective*) (EA): the offset the processor computes for a memory operand. Then it computes the physical address = DS $\times$ 16 + EA.

Mode	Example	Effective address EA
immediate	`mov ax, 5`	none: the value is in the instruction
register	`mov ax, bx`	none: the value is in a register
direct	`mov ax, [0200h]`	EA = 0200_H
register indirect	`mov ax, [bx]`	EA = BX (or SI, DI, BP)
based	`mov ax, [bx+4]`	EA = BX + 4
indexed	`mov ax, [si+4]`	EA = SI + 4
based indexed	`mov ax, [bx+si+4]`	EA = BX + SI + 4

Step by step: where does `mov ax, [bx+si+4]` read?

We suppose: DS = 2000_H, BX = 0100_H, SI = 0020_H.

1. Effective address: EA = $0100_H + 0020_H + 4 = 0124_H$.
2. Physical address: $2000_H \times 16 + 0124_H = 20000_H + 0124_H = 20124_H$.
3. The processor reads a word (two bytes): the byte at 20124_H goes into AL, and the byte at 20125_H goes into AH.

Step by step: `mov ax, [bp+2]` uses the segment SS

We suppose SS = 3000_H and BP = $FFF0_H$.

1. Effective address: EA = $FFF0_H + 2 = FFF2_H$.
2. Because we used BP, the default segment is SS, not DS.
3. Physical address: $30000_H + FFF2_H = 3FFF2_H$.

This is important in procedures: BP is used to read the parameters that are in the stack.

Watch out

The 8086 stores a word in memory with the **small byte first** (little endian): the word 1234_H is stored as 34 then 12. Also, two memory operands in the same instruction are not allowed: `mov [a], [b]` is wrong.

3.6 A first program

3.6.1 The tools

We write the program in **EMU8086**: *File* → *New* → *COM template*, then we press *Emulate* and run one instruction at a time with the *Single step* button.

Every assembly program goes through three steps:



1. **Writing**: we type the instructions in a text file (the source).
2. **Assembling**: the assembler turns each instruction into machine-code bytes, and computes the addresses of the variables and labels.
3. **Loading and running**: the system copies the bytes into memory starting at offset 100_H, sets IP = 100_H, and the processor starts.

3.6.2 The program

```
; My first 8086 program: print a message on the screen
org 100h              ; .COM program: code starts at offset 100h

    lea dx, msg       ; DX = address of the message
    mov ah, 09h       ; DOS function 09h = print a string ending with '$'
    int 21h           ; call DOS
    ret               ; end of the program, back to DOS

msg db 'Salam, I am the 8086!$'
```

screen
```
Salam, I am the 8086!
```

Line	What does it do?
1	A comment: everything after `;` is not run by the processor; it is an explanation for the reader.
2	`org 100h`: a .COM program starts at offset 100_H.
4	Put the address of the message `msg` into DX.
5	Choose DOS function 09_H: "print the string whose address is in DX".
6	`int 21h`: ask DOS to run the function chosen in AH.
7	`ret`: end of the program, back to the system.
9	Define the variable `msg` with `db` (define byte). The sign `$` marks the end of the text.

Watch out

We put the data **after** `ret`. If we put it before the instructions, the processor would try to run the letters of the message as if they were instructions!

Step by step: the program in memory after loading

These are the real bytes made by the assembler:

Offset	Bytes	Content
0100	`8D 16 09 01`	`lea dx, msg` (address 0109_H)
0104	`B4 09`	`mov ah, 09h`
0106	`CD 21`	`int 21h`
0108	`C3`	`ret`
0109	`53 61 6C 61 6D …`	the letters `S a l a m …`

The whole program is 9 bytes of instructions, and the message starts right after, at 0109_H. Notice that the assembler computed the address of `msg` by itself and put it inside the `lea` instruction.

3.6.3 Defining variables

Writing	Meaning
`x db 5`	one byte with value 5
`y dw 1234h`	one word (two bytes) with value 1234_H
`t db 14, 3, 27`	an array of 3 bytes
`buf db 10 dup(0)`	10 bytes, all 0
`s db 'Hi$'`	a string of characters

3.7 How an instruction runs inside the 8086

Let us follow the instruction `mov cx, [si]` from the program `transfer.asm` (next section). In memory, this instruction is written with the bytes 8B 0C at offset $010C_H$, and SI = 0112_H.

Step	Unit	What happens
1	BIU	computes the physical address CS $\times$ 16 + IP, and fetches the bytes 8B 0C into the queue (often before we need them).
2	EU	takes 8B from the queue: "move a word from memory to a register". Then 0C: "register CX, address in SI". IP becomes $010E_H$.
3	EU	computes the effective address: EA = SI = 0112_H.
4	BIU	computes DS $\times$ 16 + 0112_H, sends it on the address bus, with a read signal.
5	BIU	the word $ABCD_H$ arrives on the data bus (CD then AB).
6	EU	writes the result into CX: CX = $ABCD_H$. The flags do not change with `mov`.

3.8 The instruction set in detail

We use these short names in the tables: reg = register, mem = memory box, imm = immediate value.

3.8.1 Transfer instructions

Instruction	Meaning	Example
`mov dest, src`	dest ← src	`mov ax, bx`
`xchg a, b`	swap the two values	`xchg ax, bx`
`lea reg, mem`	reg gets the address of the variable	`lea si, tab`
`push src`	put on the stack	`push ax`
`pop dest`	take from the stack	`pop ax`

```
; Data transfer: MOV, XCHG, LEA
org 100h

    mov ax, 1234h     ; AX = 1234h (immediate)
    mov bl, ah        ; BL = high byte of AX = 12h
    mov bh, al        ; BH = low byte of AX  = 34h
    xchg ax, bx       ; swap AX and BX
    lea si, num       ; SI = address of num
    mov cx, [si]      ; CX = the word stored at num
    mov [res], ax     ; store AX in memory (variable res)
    ret

num dw 0ABCDh
res dw 0
```

Line	Instruction	AX	BX	CX	SI
4	`mov ax, 1234h`	1234	0000	0000	0000
5	`mov bl, ah`	1234	0012	0000	0000
6	`mov bh, al`	1234	3412	0000	0000
7	`xchg ax, bx`	3412	1234	0000	0000
8	`lea si, num`	3412	1234	0000	0112
9	`mov cx, [si]`	3412	1234	ABCD	0112
10	`mov [res], ax`	3412	1234	ABCD	0112

Line	What does it do?
5–6	Copy the two halves of AX into BX, swapped: BH = 34 and BL = 12.
8	`lea` gives the **address** 0112_{H}, not the value.
9	Read the word stored at address 0112_{H}: ABCD_{H}.
10	The only line that writes into memory: `res` becomes 3412_{H}.

3.8.2 Arithmetic instructions

Instruction	Meaning	Flags
`add d, s`	$d \leftarrow d + s$	all
`adc d, s`	$d \leftarrow d + s +$ CF (to add big numbers)	all
`sub d, s`	$d \leftarrow d - s$	all
`inc d`	$d \leftarrow d + 1$	all except CF
`dec d`	$d \leftarrow d - 1$	all except CF
`neg d`	$d \leftarrow -d$ (two's complement)	all
`cmp a, b`	computes $a - b$ **without keeping the result**	all
`mul s`	unsigned multiply (`imul` for signed)	CF, OF
`div s`	unsigned divide (`idiv` for signed)	undefined

```
1  ; Arithmetic: ADD, SUB, INC, DEC, NEG
2  org 100h
3
4      mov al, 25        ; AL = 25
5      add al, 17        ; AL = 25 + 17 = 42
6      sub al, 2         ; AL = 42 - 2  = 40
7      inc al            ; AL = 41
8      dec al            ; AL = 40
9      neg al            ; AL = -40 (two's complement)
10     mov bx, 1000      ; BX = 1000
11     add bx, 2500      ; BX = 3500
12     ret
```

Line	Instruction	Result	Decimal	CF
4	`mov al, 25`	AL = 19	25	0
5	`add al, 17`	AL = 2A	42	0
6	`sub al, 2`	AL = 28	40	0
7	`inc al`	AL = 29	41	0
8	`dec al`	AL = 28	40	0
9	`neg al`	AL = D8	-40	1
10	`mov bx, 1000`	BX = 03E8	1000	1
11	`add bx, 2500`	BX = 0DAC	3500	0

Step by step: why is -40 written $D8_H$? (two's complement)

1. Write 40 in binary: $0010\,1000_2$.
2. Flip all the bits: $1101\,0111_2$.
3. Add 1: $1101\,1000_2 = D8_H$.

Check: $40 + (-40) = 28_H + D8_H = 100_H$, and on 8 bits only 00 is left. Correct!

Multiplication and division

The processor chooses the place depending on the size of the operand:

Instruction	Operand	Operation	Result
`mul bl`	8 bits	AL × BL	AX
`mul cx`	16 bits	AX × CX	DX:AX (32 bits)
`div bl`	8 bits	AX ÷ BL	AL = quotient, AH = remainder
`div cx`	16 bits	DX:AX ÷ CX	AX = quotient, DX = remainder

```
; Multiplication and division
org 100h

    mov al, 12        ; first number
    mov bl, 11        ; second number
    mul bl            ; AX = AL * BL = 132

    mov ax, 300       ; AX = 300
    mov cx, 500
    mul cx            ; DX:AX = AX * CX = 150000

    mov ax, 47        ; dividend
    mov bl, 5         ; divisor
    div bl            ; AL = 47 / 5 = 9 , AH = remainder = 2
    ret
```

Line	What does it do?
6	$12 \times 11 = 132 = 84_{\text{H}}$: AX = 0084_{H}.
10	$300 \times 500 = 150000$, which is bigger than 65535, so it is split over two registers: DX = 0002_{H} and AX = $49\text{F}0_{\text{H}}$ (because $249\text{F}0_{\text{H}} = 150000$).
14	$47 = 5 \times 9 + 2$: AL = 09 (quotient) and AH = 02 (remainder), so AX = 0209_{H}.

Watch out

Before `div cx` we must put DX = 0 if the number to divide is only in AX; otherwise the processor takes DX as part of the number. Dividing by 0 stops the program (interrupt number 0).

Adding numbers bigger than 16 bits: `adc`

The 8086 adds 16 bits at a time. To add two 32-bit numbers, we split each number into two words (high and low). We add the low words with `add`, then the high words with `adc`, which adds the carry CF from the first addition, exactly like we carry "one" to the next column when we add with a pen.

```
1  ; Add two 32-bit numbers with a 16-bit processor
2  ; 0001FFFFh + 00000001h = 00020000h
3  org 100h
4
5      mov ax, 0FFFFh    ; low word of the 1st number
6      mov dx, 0001h     ; high word of the 1st number
7      add ax, 0001h     ; low words: FFFF + 1 = 0000, carry CF = 1
8      adc dx, 0000h     ; high words: 0001 + 0 + CF = 0002
9      ret               ; result DX:AX = 0002:0000
```

Line	Instruction	DX	AX	CF
5–6	load the first number	0001	FFFF	0
7	`add ax, 0001h`	0001	0000	**1**
8	`adc dx, 0000h`	0002	0000	0

The result is DX:AX = 0002:0000 = 00020000_{H}. If we used `add` on line 8, the carry would be lost and we would get 00010000_{H}, a wrong result.

3.8.3 Logic instructions

Instruction	Usual use	Example
`and`	switch bits off (a mask)	`and al, 0Fh`
`or`	switch bits on	`or al, 80h`
`xor`	flip bits, or clear a register	`xor cx, cx`
`not`	flip all the bits	`not al`
`test`	`and` without keeping the result (check a bit)	`test al, 1`
`shl`, `shr`	shift: multiply or divide by 2	`shl bl, 1`
`rol`, `ror`	rotate: the bit that leaves comes back on the other side	`rol al, 1`

```
1  ; Logic and shift instructions
2  org 100h
3
4      mov al, 10110110b ; AL = B6h
5      and al, 00001111b ; keep only the 4 low bits  -> 06h
6      or  al, 10000000b ; set bit 7 to 1            -> 86h
7      xor al, 11111111b ; invert all the bits       -> 79h
8      not al            ; invert again              -> 86h
9      mov bl, 5
10     shl bl, 1         ; shift left  = multiply by 2 -> 10
11     shl bl, 1         ;                               -> 20
12     shr bl, 1         ; shift right = divide by 2   -> 10
13     xor cx, cx        ; quick way to put CX = 0
14     ret
```

Line	Instruction	AL in binary	Hex
4	`mov al, 10110110b`	1011 0110	B6
5	`and al, 00001111b`	0000 0110	06
6	`or al, 10000000b`	1000 0110	86
7	`xor al, 11111111b`	0111 1001	79
8	`not al`	1000 0110	86

Line	What does it do?
5	`and` with 1 keeps the bit, and with 0 switches it off: we keep only the 4 low bits.
6	`or` with 1 switches the bit on: bit 7 is now 1.
10–12	BL: $5 \to 10 \to 20 \to 10$. A shift left multiplies by 2, and a shift right divides by 2.
13	`xor cx, cx`: every bit with itself gives 0, so CX = 0. Faster and shorter than `mov cx, 0`.

Practical uses of masks

Step by step: is the number in AL even?

A number is even if bit 0 is 0. We write `test al, 1`: this instruction computes AL · 1 without changing AL.

- If AL = 6 = $0000\,0110_2$: bit 0 is 0, the result is 0, and ZF = 1: the number is even.
- If AL = 7 = $0000\,0111_2$: bit 0 is 1, the result is 1, and ZF = 0: the number is odd.

Then we write `jz even` to jump when the number is even.

Step by step: turn a small letter into a capital letter with a mask

In the ASCII table: `'a'` = 61_H = $0110\,0001_2$ and `'A'` = 41_H = $0100\,0001_2$. The only difference is bit 5. So `and al, 11011111b` (that is `0DFh`) switches bit 5 off and turns any small letter into a capital letter, and leaves a capital letter as it is. This is another way than `sub al, 32`, which we see in the input section.

3.8.4 Flow control: jumps

Usually the processor runs the instructions in order. A **jump** changes IP, so execution goes somewhere else.

Instruction	Jumps if...	Flags
`jmp L`	always (unconditional jump)	–
`je`, `jz`	equal / result is zero	ZF = 1
`jne`, `jnz`	not equal	ZF = 0
`jg`, `jge`	greater / greater or equal (**signed**)	ZF, SF, OF
`jl`, `jle`	less / less or equal (**signed**)	SF, OF
`ja`, `jae`	above / above or equal (**unsigned**)	CF, ZF
`jb`, `jbe`	below / below or equal (**unsigned**)	CF, ZF

Everyday analogy

`cmp` is like **putting two things on a scale**, and a conditional jump is like saying: "**if** the right side goes down, go right". The scale (`cmp`) sets the flags, and the jump looks at them.

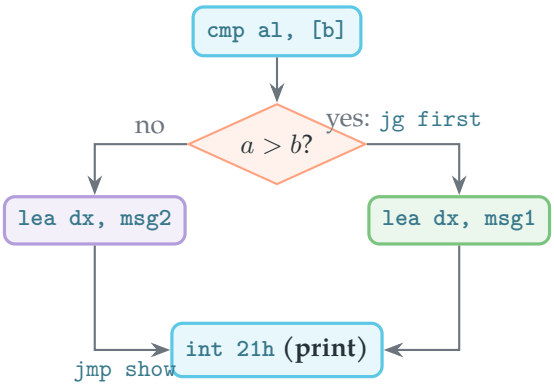
Example: if ... else

```
; IF ... ELSE : print which number is bigger
org 100h

    mov al, [a]       ; AL = a
    cmp al, [b]       ; compare a with b (computes a - b)
    jg  first         ; jump if a > b (signed)
    lea dx, msg2      ; else: b is bigger or equal
    jmp show
first:
    lea dx, msg1      ; a is bigger
show:
    mov ah, 09h       ; print the chosen message
    int 21h
    ret

a    db 7
b    db 12
msg1 db 'a is bigger$'
msg2 db 'b is bigger or equal$'
```

```
screen
b is bigger or equal
```



Line	What does it do?
5	`cmp` computes $7 - 12 = -5$ and does not keep it, but it sets the flags (SF = 1).
6	`jg`: is $a > b$? No, so we do not jump and continue at line 7.
7–8	Choose the second message, then `jmp show` is **needed** so that line 10 is not run.
12–13	The printing is shared by the two cases.

Watch out

We often forget `jmp show` on line 8! Without it, the program also runs `lea dx, msg1`, so it always prints the first message.

Signed or unsigned? `jg` or `ja`

The same bits can be read in two ways (we saw this in Chapter 2). AL = $C8_H$ is 200 unsigned, and -56 signed. So the processor has two kinds of jumps: `ja`/`jb` for unsigned numbers, and `jg`/`jl` for signed numbers.

```
; Same bits, two meanings: 200 is also -56
org 100h

    mov al, 200       ; AL = C8h
    cmp al, 100
    ja  bigger_u      ; unsigned: 200 > 100 ?
    lea dx, no_u
    jmp test_s
bigger_u:
    lea dx, yes_u
test_s:
    mov ah, 09h
    int 21h
    mov al, 200
    cmp al, 100
    jg  bigger_s      ; signed: -56 > 100 ?
    lea dx, no_s
    jmp fin
bigger_s:
    lea dx, yes_s
fin:
    mov ah, 09h
    int 21h
    ret

yes_u db 'unsigned: 200 > 100', 13, 10, '$'
no_u  db 'unsigned: 200 <= 100', 13, 10, '$'
yes_s db 'signed: -56 > 100$'
no_s  db 'signed: -56 <= 100$'
```

screen

```
unsigned: 200 > 100
signed: -56 <= 100
```

Line	What does it do?
4–6	`ja`: compare 200 with 100 as positive numbers. $200 > 100$, so we jump.
14–16	`jg`: the same bits $C8_H$, but read as signed: $-56 > 100$? No, so we do not jump.

Watch out

Choose the jump from the **kind of data**: ages, addresses, character codes, sizes are unsigned (`ja`, `jb`). Temperatures, balances, differences are signed (`jg`, `jl`).

From the algorithm to assembly

Each control structure from the algorithms course has a fixed "translation" in assembly. The idea is always the same: we **compare** with `cmp`, then we **jump** depending on the result.

Structure	In C	In assembly

if	`if (a == b) x = 1;`	`cmp a, b`
		`jne end`
		`mov x, 1`
		`end:`

while	`while (cx > 0) { ... }`	`L: cmp cx, 0`
		`jle end`
		`...`
		`jmp L`
		`end:`

for	`for (cx = 10; cx > 0; cx--)`	`mov cx, 10`
		`L: ...`
		`loop L`

Watch out

In the translation of `if`, we jump with the **opposite** condition (`jne` instead of `je`). We tell the processor: "if they are not equal, skip the assignment". This makes the program shorter.

3.8.5 Loops and repetition

Definition

`loop L` is like two instructions: `dec cx`, then "if CX $\neq 0$, jump to L". So CX is the loop counter.

The sum of the numbers from 1 to 10

```
1  ; FOR loop with LOOP: sum = 1 + 2 + ... + 10
2  org 100h
3
4      mov ax, 0          ; sum = 0
5      mov cx, 10         ; CX = number of turns (and current value)
6  again:
7      add ax, cx         ; sum = sum + CX
8      loop again         ; CX = CX - 1 ; if CX <> 0 jump to again
9      mov [sum], ax      ; sum = 55
10     ret
11
12 sum dw 0
```

Turn	1	2	3	4	5	6	7	8	9	10
CX before `add`	10	9	8	7	6	5	4	3	2	1
AX after `add`	10	19	27	34	40	45	49	52	54	55

After the tenth turn CX = 0, so the loop stops, and AX = 0037_H = 55. Notice that we add backwards: $10 + 9 + \cdots + 1$, and the result is the same.

Watch out

If we enter the loop with CX = 0, `loop` first decreases CX, which becomes $FFFF_H$, and then repeats 65535 more times! To avoid this, we put `jcxz end` before the loop: "if CX = 0, jump to the end".

Finding the biggest value in an array

```
; WHILE loop: find the biggest value in an array
org 100h

    lea si, tab       ; SI points to the first element
    mov cx, 6         ; 6 elements
    mov al, [si]      ; max = first element
next:
    cmp [si], al      ; compare current element with max
    jbe skip          ; if element <= max, keep max
    mov al, [si]      ; else max = element
skip:
    inc si            ; go to the next element
    loop next
    mov [max], al
    ret

tab db 14, 3, 27, 9, 31, 8
max db 0
```

SI points to	Element	Element > AL?	AL after	CX after `loop`
tab[0]	14	no (equal)	14	5
tab[1]	3	no	14	4
tab[2]	27	yes	27	3
tab[3]	9	no	27	2
tab[4]	31	yes	**31**	1
tab[5]	8	no	31	0

The result: AL = $1F_H$ = 31, stored in `max`. We used `jbe` because the values are **unsigned**.

Repetition with the `rep` prefix

String instructions work on DS:SI (the source) and ES:DI (the destination), and the prefix `rep` repeats them CX times.

```
; Repetition with REP: copy a string with one instruction
org 100h

    lea si, src       ; SI = source
    lea di, dst       ; DI = destination
    mov cx, 6         ; number of bytes to copy
    cld               ; direction = forward (SI and DI increase)
    rep movsb         ; repeat CX times: [DI] = [SI], SI++, DI++
    lea dx, dst
    mov ah, 09h       ; print the copy
    int 21h
    ret

src db 'Hello$'
dst db 6 dup(0)
```

```
screen
Hello
```

Line	What does it do?
7	`cld`: set DF = 0, so SI and DI increase after each copy.
8	`rep movsb` copies one byte from [SI] to [DI], and repeats 6 times: the five letters and `$`.

3.8.6 Generating machine code

The assembler turns each instruction into bytes. These are the real bytes of the program `machine.asm`:

```
; Small program to study its machine code
org 100h

    mov ax, 1234h
    mov bx, ax
    add al, 5
    mov [bx], al
    inc cx
    jmp fin
    nop
fin:
    ret
```

Address	Machine code (hex)	Instruction
0100	`B8 34 12`	`mov ax, 1234h`
0103	`89 C3`	`mov bx, ax`
0105	`04 05`	`add al, 5`
0107	`88 07`	`mov [bx], al`
0109	`41`	`inc cx`
010A	`EB 01`	`jmp fin`
010C	`90`	`nop`
010D	`C3`	`ret`

Step by step: decoding B8 34 12

1. B8 = $1011\,1000_2$: the first five bits 10111 mean "`mov` an immediate value into a 16-bit register", and the last three bits 000 are the number of the register AX.
2. 34 12: the value 1234_H, written with the small byte first (little endian).

Step by step: decoding 89 C3 (`mov bx, ax`)

The first byte 89 = $100010\,0\,1_2$:

- 100010: the opcode of `mov` between a register and a register or memory.
- d = 0: the register named in the *reg* field is the **source**.
- w = 1: we work on 16 bits (if it were 0 we would work on 8 bits).

The second byte C3 = $11\,000\,011_2$ is called ModR/M:



mod: register reg = AX r/m = BX

So: move AX (the source) into BX.

Number	000	001	010	011	100	101	110	111
w = 1	AX	CX	DX	BX	SP	BP	SI	DI
w = 0	AL	CL	DL	BL	AH	CH	DH	BH

Example: 88 07 and EB 01

88 = $100010\,0\,0_2$: `mov` on 8 bits. 07 = $00\,000\,111_2$: mod = 00 (memory without offset), reg = 000 (AL), r/m = 111 ([BX]). So `mov [bx], al`.

EB 01: a short jump, and the offset is **relative**: after reading `jmp`, IP = $010C_H$, and we add 1 to reach $010D_H$ (`fin`). This is how we jumped over `nop`.

3.9 Input and output

3.9.1 Ports: `in` and `out`

The processor talks with devices through numbered **ports** (Fr. *ports d'entrée/sortie*), from 0 to $FFFF_H$.

- `in al, 60h`: read one byte from port 60_H (the keyboard in a PC).
- `out 61h, al`: send AL to port 61_H (the speaker).

Normal programs rarely use ports directly; they ask the operating system through interrupts.

3.9.2 DOS services: the `int 21h` interrupt

AH	Function	Inputs and outputs
01_H	read a key from the keyboard and show it	output: the character in AL
02_H	print a character	input: the character in DL
09_H	print a string ending with `$`	input: its address in DX
$4C_H$	end the program	AL = exit code

```
1  ; Read a lowercase letter and print it in uppercase
2  org 100h
3
4      lea dx, ask
5      mov ah, 09h       ; print the question
6      int 21h
7      mov ah, 01h       ; DOS: read one key -> AL (with echo)
8      int 21h
9      sub al, 32        ; 'a' (97) - 32 = 'A' (65)
10     mov bl, al        ; keep the letter in BL
11     lea dx, ans
12     mov ah, 09h
13     int 21h
14     mov dl, bl        ; DOS function 02h prints the character in DL
15     mov ah, 02h
16     int 21h
17     ret
18
19 ask db 'Type a letter: $'
20 ans db 13, 10, 'Uppercase: $'
```

```
screen
Type a letter: m
Uppercase: M
```

Line	What does it do?
7–8	Wait for the user to press a key (here `m`); its ASCII code arrives in AL: 109.
9	In the ASCII table, a capital letter is 32 less than the small one: $109 - 32 = 77$, which is the code of `'M'`.
10	Keep the letter in BL, because the printing on line 13 changes AL.
14–16	Function 02_H prints the character that is in DL.
20	`13, 10`: back to the start of the line, then a new line.

3.10 The stack

Definition

The **stack** (Fr. *pile*): a zone of memory where we put values and take them back with the rule **last in, first out** (LIFO). Its top is given by SS:SP.

Everyday analogy

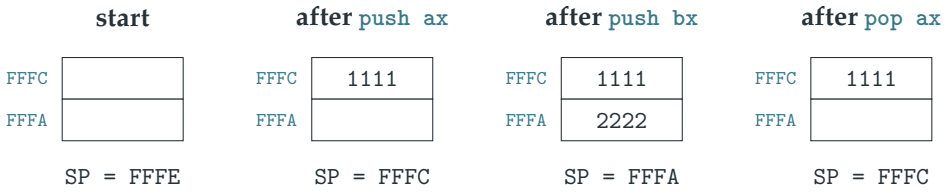
The stack is like a **pile of plates**: we put the new plate on top of the pile, and we always take the plate on top.

- `push ax`: SP ← SP - 2, then write AX at [SP].
- `pop ax`: read [SP] into AX, then SP ← SP + 2.

Watch out

In the 8086 the stack **grows towards small addresses**: each `push` decreases SP. And we always put and take **words** (16 bits).

```
1  ; The stack: PUSH and POP
2  org 100h
3
4      mov ax, 1111h
5      mov bx, 2222h
6      push ax           ; SP = SP - 2 ; [SP] = AX
7      push bx           ; SP = SP - 2 ; [SP] = BX
8      pop ax            ; AX = [SP]   ; SP = SP + 2  -> AX = 2222h
9      pop bx            ; BX = [SP]   ; SP = SP + 2  -> BX = 1111h
10     ret               ; AX and BX have been swapped
```



Line	Instruction	AX	BX	SP
4–5	`mov ax, mov bx`	1111	2222	FFFE
6	`push ax`	1111	2222	FFFC
7	`push bx`	1111	2222	FFFA
8	`pop ax`	2222	2222	FFFC
9	`pop bx`	2222	1111	FFFE

Because the last value in (2222_H) is the first value out, AX and BX have swapped their values.

3.10.1 Why do we need a stack?

1. **To keep return addresses**: when we call a procedure, the processor keeps in the stack where it must come back.
2. **To keep registers for a while**: a procedure needs CX, but CX is used by a loop? We write `push cx` at its start and `pop cx` at its end, so the loop counter is not lost.
3. **To pass parameters** to procedures (we see it soon).
4. **Interrupts**: the processor keeps its state there before going to the interrupt handler.

Watch out

The number of `pop` must equal the number of `push`, in the opposite order. `push ax` then `push bx` must be matched by `pop bx` then `pop ax`. If you forget one `pop` inside a procedure, `ret` takes a wrong value as the return address, and the program jumps to a random place.

3.11 Functions and procedures

Definition

A **procedure** (Fr. *procédure, sous-programme*): a group of instructions with a name. We call it with `call` and come back from it with `ret`. We write it once and use it many times.

3.11.1 How does `ret` know where to go back?

- `call P`: puts on the stack **the address of the instruction after it** (IP), then jumps to P.
- `ret`: takes this address from the stack and puts it into IP, so execution goes back after `call`.

Step by step: the two calls in the next program (real addresses)

1. The first `call print_num` is at offset 0103_H and is 3 bytes long, so the next instruction is at 0106_H. The processor puts 0106_H on the stack, then jumps to `print_num`.
2. At the end of the procedure, `ret` takes 0106_H from the stack: execution goes back to `mov ax, 7`.
3. The second call is at 0109_H, so it puts $010C_H$, and this time `ret` goes back to $010C_H$.

The same procedure goes back to two different places, because the return address is kept in the stack each time.

3.11.2 A procedure that prints a number in decimal

`int 21h` only prints characters. To print the number 2026, we must take out its digits one by one:

Step by step: taking out the digits of 2026

Operation	Quotient (AX)	Remainder (DX)	Stack
$2026 \div 10$	202	6	6
$202 \div 10$	20	2	6, 2
$20 \div 10$	2	0	6, 2, 0
$2 \div 10$	0 (we stop)	2	6, 2, 0, 2

The digits come out backwards $(6, 2, 0, 2)$. We put them on the stack, then take them back: they come out in the right order 2, 0, 2, 6.

```
; A procedure that prints the number in AX in decimal
org 100h

    mov ax, 2026
    call print_num    ; print 2026
    mov ax, 7
    call print_num    ; print 7
    ret

print_num proc        ; input: AX = number to print
    mov bx, 10        ; we divide by 10
    mov cx, 0         ; CX = number of digits
divide:
    mov dx, 0         ; DX:AX / BX needs DX = 0
    div bx            ; AX = AX / 10 , DX = last digit
    push dx           ; keep the digit on the stack
    inc cx
    cmp ax, 0
    jne divide        ; continue while AX <> 0
show:
    pop dx            ; digits come back in the right order
    add dl, '0'       ; digit -> ASCII character
    mov ah, 02h
    int 21h
    loop show
    mov dl, ' '       ; a space after the number
    int 21h
    ret
print_num endp
```

screen

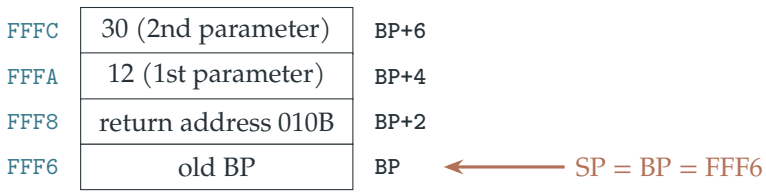
```
2026 7
```

Line	What does it do?
4–5	Put the number in AX (the parameter), then call the procedure.
10	`proc` and `endp` mark the start and the end of the procedure.
14	DX = 0 is needed before `div bx` (a 16-bit division divides DX:AX).
16–17	Put the digit on the stack, and count the digits in CX.
21–25	CX times: take a digit, turn it into a character by adding `'0'` (48), and print it.
28	`ret` goes back to line 6 the first time, and to line 8 the second time.

3.11.3 Passing parameters on the stack

```
1  ; Passing parameters on the stack
2  org 100h
3
4      mov ax, 30
5      push ax           ; 2nd parameter
6      mov ax, 12
7      push ax           ; 1st parameter
8      call addtwo       ; result in AX
9      mov [res], ax     ; res = 42
10     ret
11
12 addtwo proc
13     push bp           ; save BP
14     mov bp, sp        ; BP = top of the stack
15     mov ax, [bp+4]    ; 1st parameter (12)
16     add ax, [bp+6]    ; + 2nd parameter (30)
17     pop bp            ; restore BP
18     ret 4             ; return and remove the 2 parameters
19 addtwo endp
20
21 res dw 0
```

After `mov bp, sp` on line 14, the stack looks like this (values from the real run):



Line	What does it do?
4–7	Put the two parameters on the stack: SP goes from $FFFE_H$ to $FFFA_H$.
8	`call` puts the return address $010B_H$: SP = $FFF8_H$.
13–14	Save BP, then make BP stay fixed on the top of the stack ($FFF6_H$).
15–16	The first parameter is at [BP+4], the second at [BP+6]: $12 + 30 = 42$.
18	`ret 4`: go back, then remove 4 bytes (the two parameters) from the stack. SP goes back to $FFFE_H$.

3.12 Interrupts

Definition

An **interrupt** (Fr. *interruption*): an event that stops the current program for a moment, so the processor runs a special program (the **interrupt handler**), then comes back to continue where it stopped.

Everyday analogy

You are reading a book and the **doorbell** rings. You put a bookmark in the page, open the door, then come back and continue reading from the same page. The bell is the interrupt, and the bookmark is what the processor keeps in the stack.

3.12.1 Kinds of interrupts

Kind	Who causes it?	Examples
hardware	an outside device sends a signal on the INTR or NMI pin	a key press, the clock, the printer is ready
software	the instruction `int n` in the program	`int 21h` for DOS services
exceptions	the processor itself, after an error	division by zero (interrupt 0)

3.12.2 The interrupt vector table

At the start of memory (from 00000_{H} to 003FF_{H}) there is a table of 256 entries. Each interrupt has 4 bytes: the offset, then the segment, of its handler.

Definition

The vector of interrupt number n is at address: $4 \times n$

Example

The vector of `int 21h`: $21_{\text{H}} \times 4 = 33 \times 4 = 132 = 84_{\text{H}}$. So the address of the DOS handler is kept from 00084_{H} to 00087_{H}.

3.12.3 What does the processor do on `int n`?

1. It puts FLAGS on the stack, then switches IF and TF off.
2. It puts CS, then IP, on the stack (the bookmark in the book).
3. It reads the new IP and CS from the entry $n \times 4$, so it jumps to the interrupt handler.
4. At the end of the handler, `iret` takes IP, CS and FLAGS back from the stack, and the program continues.

3.12.4 A hardware interrupt, step by step

Step by step: you press a key while a program runs

1. The keyboard sends a signal to the **interrupt controller** (Fr. *contrôleur d'interruptions*) (PIC 8259).
2. The controller sends a signal on the INTR input of the processor.
3. If IF = 1, the processor first finishes the **current instruction**, then accepts the interrupt.
4. The controller gives the processor the interrupt number (09_H for the keyboard).
5. The processor saves FLAGS, CS and IP, then jumps to the address written at $4 \times 9 = 24_H$.
6. The handler reads the key from port 60_H, keeps it, then runs `iret`.
7. The program continues from where it stopped, without "noticing" anything.

Definition

- `cli` (clear IF) blocks hardware interrupts for a while, and `sti` (set IF) allows them again. We use them when a sensitive part of the program must not be interrupted.
- The NMI input (Non Maskable Interrupt): an interrupt that **cannot be blocked**, used for serious errors such as a memory fault or a power failure.

3.12.5 A program: our own interrupt

```
; Install our own interrupt handler for INT 60h
org 100h

    mov ax, 0
    mov es, ax            ; ES = 0 : the interrupt vector table
    lea ax, handler
    mov es:[60h*4], ax    ; offset of the handler
    mov es:[60h*4+2], cs  ; segment of the handler
    mov bl, 'A'
    int 60h               ; call our interrupt
    int 60h               ; call it again
    ret

handler:                  ; prints BL, then BL = next letter
    mov dl, bl
    mov ah, 02h
    int 21h
    inc bl
    iret                  ; return from interrupt (restores IP, CS, FLAGS)
```

```
screen
AB
```

Line	What does it do?
4–5	ES = 0: we want to write in the vector table, which is in segment 0.
6–8	Write the address of `handler` in entry $60_H \times 4 = 180_H$: the offset, then the segment.
10–11	Each `int 60h` runs `handler`: the first time it prints `A`, the second time `B`.
19	`iret` and not `ret`, because the interrupt also put FLAGS on the stack.

3.13 Exercises

Exercise 3.1

Compute the physical address in each case: (a) DS = 3000H, BX = 0050H with `mov al, [bx]`. (b) SS = 4000H, SP = FFFEH.

Solution

(a) $30000_H + 0050_H = 30050_H$. (b) $40000_H + FFFE_H = 4FFFE_H$.

Exercise 3.2

What is AX after these instructions? `mov ax, 0F0Fh`, then `and ax, 00FFh`, then `shl ax, 1`.

Solution

After `and`: $000F_H$. After the shift (multiply by 2): $001E_H = 30$.

Exercise 3.3

Write a program that computes $5! = 1 \times 2 \times 3 \times 4 \times 5$ with the `loop` instruction, and puts the result in AX.

Solution

```
1  ; Exercise: 5! with LOOP
2  org 100h
3
4      mov ax, 1          ; result = 1
5      mov cx, 5          ; multiply by 5, 4, 3, 2, 1
6  again:
7      mul cx             ; DX:AX = AX * CX
8      loop again
9      ret                ; AX = 120
```

The result: AX = 0078 = 120 (tested in the emulator).

Exercise 3.4

After `mov sp, 0100h`, then `push ax`, then `push bx`, then `pop cx`: what is SP, and which value is in CX?

Solution

After the two pushes SP = $00FC_H$, and after `pop` SP = $00FE_H$. CX gets the last value in: the value of BX.

Exercise 3.5

What is the vector address of interrupt number 10_H (BIOS screen services)?

Solution

$10_H \times 4 = 16 \times 4 = 64 = 40_H$: the bytes from 00040_H to 00043_H.

Exercise 3.6

Decode the instruction 89 D8.

Solution

89: `mov` on 16 bits, d = 0. D8 = $11\,011\,000_2$: mod = 11 (register), reg = 011 (BX, the source), r/m = 000 (AX). So: `mov ax, bx`.

Chapter 4

Case Study: 32-bit and 64-bit Processors

Summary: what will we learn in this chapter?

- Why we moved from 16 bits to 32 and then 64 bits, and what changed.
- How the architecture grew from IA-32 to AMD64.
- The registers of modern processors, and flat addressing.
- New instructions: SSE and AVX, which compute several numbers at the same time.
- The pipeline: how the processor runs several instructions at the same time.

Watch out

The programs of this chapter are written in **NASM** syntax for **Linux**, and we really ran them on an Intel Core i7-8665U processor (4 cores). To assemble and run: `nasm -f elf64 prog.asm`, then `ld prog.o -o prog`, then `./prog`.

4.1 From 16 to 32, then 64 bits

Definition

We say a processor is an n**-bit** processor when its **general registers** are n bits wide: this is the size of the number it computes in one instruction, and the size of the address it can keep in a register.

4.1.1 The main differences

	16 bits (8086)	**32 bits (IA-32)**	**64 bits (x86-64)**
biggest unsigned number	$2^{16} - 1 = 65\,535$	$2^{32} - 1 \approx 4.3$ billion	$2^{64} - 1 \approx 1.8 \times 10^{19}$
max memory	1 MB (with segments)	4 GB	256 TB (48 bits in practice)
general registers	8	8	**16**
register names	AX, BX, ...	EAX, EBX, ...	RAX, RBX, ..., R15
addressing	segmented	flat (in practice)	flat

4.1.2 Why do we need more than 32 bits?

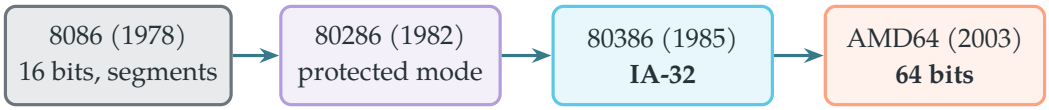
1. **Memory**: with a 32-bit address we cannot use more than 4 GB. Today's computers have 16 or 32 GB, and servers have hundreds of gigabytes.
2. **Performance**: a 64-bit register computes a big number in one instruction, while a 32-bit processor needs at least two instructions (like `add` and `adc` in Chapter 3). And twice as many registers (16 instead of 8) means fewer trips to memory.
3. **Compatibility**: a 64-bit processor also runs old 32-bit programs without any change.

Step by step: what does ``4 GB'' mean exactly?

1. A 32-bit address can take 2^{32} different values.
2. $2^{32} = 2^2 \times 2^{30} = 4 \times 2^{30}$.
3. And 2^{30} bytes $= 1$ GB, so the maximum size is 4 GB.

With 48 bits: $2^{48} = 2^8 \times 2^{40} = 256$ TB, that is 65536 times bigger!

4.2 How the architecture grew: from IA-32 to AMD64



- **IA-32** (Intel Architecture 32 bits): it started with the 80386. Registers became 32 bits wide (EAX: the E means *Extended*), and the **protected mode** appeared: it stops programs from writing in each other's memory.
- **AMD64**: the surprise is that AMD (not Intel) designed the 64-bit extension in 2003, while keeping compatibility. Intel adopted it later as Intel 64. Today we call it x86-64.

4.2.1 The working modes of the processor

Mode	Description	When?
real mode	behaves like an 8086: 16 bits and 1 MB	when the computer starts, before the operating system
protected mode	32 bits, memory protection	32-bit systems
long mode	64 bits, with a sub-mode to run 32-bit programs	all modern systems

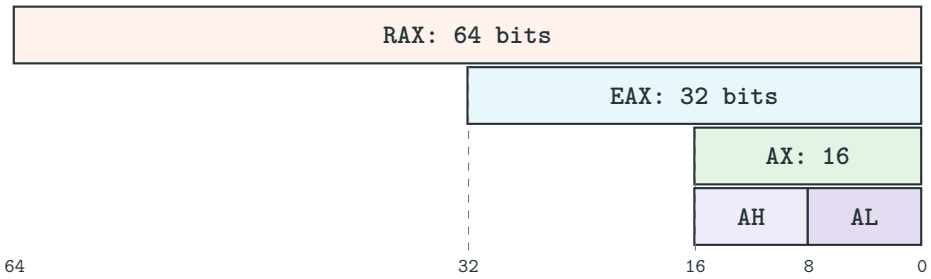
Did you know?

Every modern computer wakes up "thinking" it is an 8086! It starts in real mode, then the operating system moves it to protected mode, then to long mode. This is how the architecture stays compatible since 1978.

4.3 Registers in 32-bit and 64-bit processors

4.3.1 Nested registers

Each new register **contains** the old register in its right half (the low bits):



64 bits	32 bits	16 bits	8 bits	Usual use
RAX	EAX	AX	AL	value returned by a function
RBX	EBX	BX	BL	general
RCX	ECX	CX	CL	counter, 4th parameter
RDX	EDX	DX	DL	3rd parameter
RSI	ESI	SI	SIL	2nd parameter
RDI	EDI	DI	DIL	1st parameter
RSP	ESP	SP	SPL	top of the stack
RBP	EBP	BP	BPL	base of the stack frame
R8 ... R15	R8D ... R15D	R8W ...	R8B ...	**new in 64 bits**

- **RIP**: the instruction pointer (it was IP, then EIP).
- **RFLAGS**: the flags register; ZF, CF, SF, OF are in the same place as in the 8086.
- **Segment registers**: they still exist (CS, DS...) but their base is 0 in 64-bit mode, except FS and GS, which the system uses for data that belongs to each thread.

4.3.2 A test: what happens to the other bits?

```
; RAX, EAX, AX, AL: what happens to the other bits?
section .text
global _start
_start:
    mov rax, 0x1122334455667788
    call print_hex          ; the full 64-bit value

    mov rax, 0x1122334455667788
    mov eax, 0xAABBCCDD     ; write 32 bits: the high half becomes 0!
    call print_hex

    mov rax, 0x1122334455667788
    mov ax, 0x1234          ; write 16 bits: the rest is kept
    call print_hex

    mov rax, 0x1122334455667788
    mov al, 0xFF            ; write 8 bits: the rest is kept
    call print_hex

    mov rax, 60             ; exit
    xor rdi, rdi
    syscall

print_hex:                  ; print RAX as 16 hex digits
    mov rcx, 16             ; 16 digits of 4 bits
    lea rdi, [rel buf]
next_digit:
    rol rax, 4              ; bring the next 4 bits to the right
    mov rdx, rax
    and rdx, 0xF            ; keep only these 4 bits
    mov dl, [digits + rdx]  ; 0..15 -> '0'..'F'
    mov [rdi], dl
    inc rdi
    dec rcx
    jnz next_digit
    mov byte [rdi], 10      ; new line
    mov rax, 1              ; write(1, buf, 17)
    mov rdi, 1
    lea rsi, [rel buf]
    mov rdx, 17
    syscall
    ret

section .data
digits  db "0123456789ABCDEF"

section .bss
buf     resb 17
```

```
screen
1122334455667788
00000000AABBCCDD
1122334455661234
11223344556677FF
```

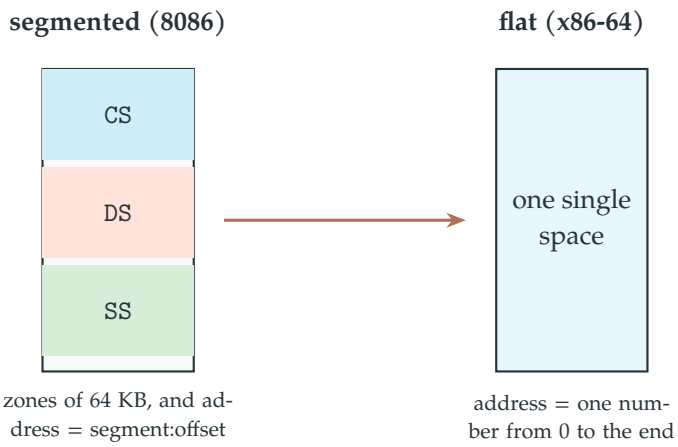
Line	What does it do?
5–6	Put a 64-bit number in RAX and show it: 1122334455667788.
9	Writing EAX (32 bits) **clears** the high half: 00000000AABBCCDD.
13	Writing AX (16 bits) keeps the other bits: ...5566**1234**.
17	Writing AL changes only the last byte: ...77**FF**.
24–42	A procedure that shows RAX in hexadecimal: at each turn it rotates 4 bits, keeps them with the mask `0xF`, and turns them into a character.

Watch out

A strange but important rule: in 64-bit mode, **every write to a 32-bit register clears the high bits**, but a write to 8 or 16 bits does not. AMD designed this rule to make the processor faster.

4.4 Advanced addressing modes

4.4.1 From segmented addressing to flat addressing



In the **flat memory model** (Fr. *modèle mémoire plat*), the program sees memory **as one long strip** of bytes, numbered from 0. We do not need to compute "segment × 16 + offset": the address is just a number in a 64-bit register.

Everyday analogy

Segmented addressing is like saying "building 12, floor 3". Flat addressing is like giving every flat in the town one unique number: "flat number 73 502". Much simpler, as long as we have big enough numbers.

4.4.2 The general form of an address

Definition

$$\text{address} = \text{base} + \text{index} \times \text{scale} + \text{displacement}$$

base and index: any two registers. scale: 1, 2, 4 or 8. displacement: a fixed number.

Writing (NASM)	Use
`[rbx]`	base only: a pointer
`[rbx + 16]`	base + displacement: a field in a structure (struct)
`[tab + rcx*8]`	index × scale + displacement: element i of an array of 8-byte numbers
`[rbx + rcx*4 + 8]`	everything together
`[rel msg]`	relative to RIP: special to 64 bits

Step by step: where does `[tab + rcx*8]` read when RCX = 3?

1. Each element of `tab` is a 64-bit number = 8 bytes (`dq`).
2. Element number 3 (the fourth one, because we start at 0) is $3 \times 8 = 24$ bytes from the start.
3. If `tab` is at address 402000_{H}, the address is $402000_{\text{H}} + 24 = 402018_{\text{H}}$.

The scale saves us from multiplying the index ourselves: the processor does it inside the instruction.

```
; Sum of an array of 64-bit numbers: base + index*scale addressing
section .text
global _start
_start:
    xor rax, rax                ; sum = 0
    xor rcx, rcx                ; i = 0
again:
    add rax, [tab + rcx*8]      ; sum += tab[i]   (8 bytes per element)
    inc rcx                     ; i++
    cmp rcx, 5
    jl  again                   ; while i < 5
    call print_dec              ; print the sum

    mov rax, 60
    xor rdi, rdi
    syscall

print_dec:                      ; print RAX in decimal
    lea rsi, [rel dbuf + 20]    ; fill the buffer from the end
    mov byte [rsi], 10          ; new line
    mov rbx, 10
    mov rcx, 1                  ; length (the new line)
divide:
    xor rdx, rdx
    div rbx                     ; RAX = RAX / 10, RDX = digit
    add dl, 48                  ; digit -> ASCII ('0' = 48)
    dec rsi
    mov [rsi], dl
    inc rcx
    test rax, rax
    jnz divide
    mov rax, 1                  ; write(1, rsi, rcx)
    mov rdi, 1
    mov rdx, rcx
    syscall
    ret

section .data
tab     dq 1000000000000, 2500, 30000000000, 7, 12345

section .bss
dbuf    resb 21
```

screen

```
1030000014852
```

Line	What does it do?
5–6	`xor` of a register with itself = 0: the sum and the index start at 0.
8	Addressing **with a scale**: read element i of the array, and add it to RAX.
9–11	`i++`, then "while i < 5" go back. This is a `while` loop, as in Chapter 3.
18–36	The same method as in Chapter 3 to print a number in decimal (divide by 10), but with 64-bit registers.
39	`dq` (define quad-word): 64-bit numbers. The first number (10^{12}) does not fit in 32 bits.

4.4.3 RIP-relative addressing

In 64-bit mode we often write `lea rsi, [rel buf]`: the address is computed **relative to the current instruction** (RIP + distance). The benefit: the program works wherever the system puts it in memory. This is important for security (the system moves programs at each run so attackers cannot guess the addresses).

4.5 The instruction set

4.5.1 The same instruction on 16, 32 and 64 bits

We assembled the program below; these are the real machine-code bytes:

```
1  ; The same addition on 16, 32 and 64 bits
2  section .text
3  global _start
4  _start:
5      add ax, bx          ; 16 bits
6      add eax, ebx        ; 32 bits
7      add rax, rbx        ; 64 bits
8      add r8, r9          ; new 64-bit registers
9      jmp fin             ; a short jump
10 fin:
11     mov rax, 60
12     xor rdi, rdi
13     syscall
```

Instruction	Machine code	Note
`add ax, bx`	`66 01 D8`	prefix 66: "careful, 16 bits"
`add eax, ebx`	`01 D8`	32 bits is the default size
`add rax, rbx`	`48 01 D8`	prefix REX.W = 48: "64 bits"
`add r8, r9`	`4D 01 C8`	REX = 4D: 64 bits + new registers

Step by step: why is 01 D8 in the first three lines?

1. 01 is the opcode "add a register to a register or memory". It is the same since the 8086.
2. D8 = $11\,011\,000_2$ is the ModR/M byte we saw in Chapter 3: mod = 11 (register), reg = 011 (BX), r/m = 000 (AX).
3. The size (16, 32 or 64) is not written in the instruction itself, but in a **prefix** before it. This is how the old instructions stayed valid, and the new sizes were added "from the outside".
4. In 4D = $0100\,1101_2$: 0100 means "REX prefix", then the bit W = 1 (64 bits), and two bits that add 8 to the two register numbers, so AX becomes R8 and BX becomes R9.

4.5.2 Comparison: a 32-bit program and a 64-bit program

In 32 bits:

```
; The same program in 32-bit Linux (NASM syntax)
section .data
msg     db "Hello from 32-bit mode!", 10
len     equ $ - msg

section .text
global _start
_start:
    mov eax, 4          ; system call 4 = write (32-bit numbers differ!)
    mov ebx, 1          ; stdout
    mov ecx, msg        ; address of the text
    mov edx, len        ; number of bytes
    int 0x80            ; 32-bit way to call Linux

    mov eax, 1          ; system call 1 = exit
    mov ebx, 0
    int 0x80
```

screen
```
Hello from 32-bit mode!
```

In 64 bits:

```
; Hello in 64-bit Linux (NASM syntax)
section .data
msg     db "Hello from 64-bit mode!", 10    ; text + new line
len     equ $ - msg                          ; length computed by NASM

section .text
global _start
_start:
    mov rax, 1          ; system call 1 = write
    mov rdi, 1          ; file 1 = the screen (stdout)
    mov rsi, msg        ; address of the text
    mov rdx, len        ; number of bytes
    syscall             ; ask Linux to do it

    mov rax, 60         ; system call 60 = exit
    mov rdi, 0          ; exit code 0 = success
    syscall
```

screen

```
Hello from 64-bit mode!
```

	32 bits	64 bits
asking the system for a service	`int 0x80` (an interrupt)	`syscall` (a special, faster instruction)
number of the *write* service	4 in EAX	1 in RAX
parameters	EBX, ECX, EDX	RDI, RSI, RDX
size of an address	4 bytes	8 bytes

Did you know?

We ran the two programs on the same 64-bit computer. The 32-bit program worked without any change: this is the compatibility sub-mode inside long mode.

4.5.3 What does the compiler write? From C to assembly

We wrote a function in C, and asked the compiler gcc to give us the assembly it produces, with the command `gcc -O1 -S -masm=intel sum.c`:

```c
/* compiled by gcc for x86-64 */
long sum(long *t, long n)
{
    long s = 0;
    for (long i = 0; i < n; i++)
        s += t[i];
    return s;
}
```

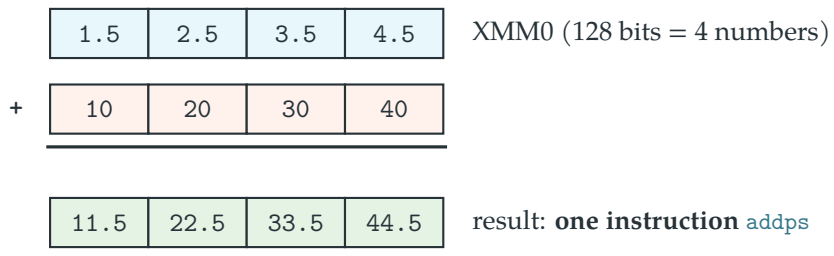
```asm
sum:
    test rsi, rsi
    jle .L4
    mov rax, rdi
    lea rcx, [rdi+rsi*8]
    mov edx, 0
.L3:
    add rdx, QWORD PTR [rax]
    add rax, 8
    cmp rax, rcx
    jne .L3
.L1:
    mov rax, rdx
    ret
.L4:
    mov edx, 0
    jmp .L1
```

Line	What does it do?
1	By the Linux 64-bit rules: the first parameter (`t`) arrives in RDI, and the second one (`n`) in RSI.
2–3	If $n \leq 0$ we jump to `.L4`: the result is 0.
5	`lea` computes the address of the end of the array $t + n \times 8$ **without reading memory**.
8–11	The loop: add the element, move 8 bytes forward, and stop at the end. The compiler used a pointer instead of the index `i`.
13–14	The result is always returned in RAX.

4.5.4 SSE and AVX: computing several numbers at once

Definition

SIMD (Fr. *une instruction, données multiples*) (Single Instruction, Multiple Data): one instruction applies the same operation to **several numbers at the same time**.



Extension	Year	Registers	Size	floats at once
SSE	1999	XMM0–XMM15	128 bits	4
AVX	2011	YMM0–YMM15	256 bits	8
AVX-512	2016	ZMM0–ZMM31	512 bits	16

```c
/* SSE: add 4 floats with one instruction */
#include <stdio.h>
#include <xmmintrin.h>

int main(void)
{
    float a[4] = {1.5, 2.5, 3.5, 4.5};
    float b[4] = {10, 20, 30, 40};
    float c[4];

    __m128 x = _mm_loadu_ps(a);   /* load 4 floats into XMM */
    __m128 y = _mm_loadu_ps(b);
    __m128 z = _mm_add_ps(x, y);  /* ONE instruction: addps */
    _mm_storeu_ps(c, z);

    printf("%.1f %.1f %.1f %.1f\n", c[0], c[1], c[2], c[3]);
    return 0;
}
```

screen
```
11.5 22.5 33.5 44.5
```

When we open the compiled program, we really find the instruction:

```
addps  xmm0, XMMWORD PTR [rbp-0x50]   ; 4 additions at once
```

And the AVX version adds 8 numbers with the instruction `vaddps ymm0, ymm0, […]`:

```c
/* AVX: add 8 floats with one instruction */
#include <stdio.h>
#include <immintrin.h>

int main(void)
{
    float a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    float b[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    float c[8];

    __m256 x = _mm256_loadu_ps(a);   /* 8 floats in a YMM register */
    __m256 y = _mm256_loadu_ps(b);
    _mm256_storeu_ps(c, _mm256_add_ps(x, y));   /* vaddps */

    for (int i = 0; i < 8; i++)
        printf("%.0f ", c[i]);
    printf("\n");
    return 0;
}
```

screen
```
11 22 33 44 55 66 77 88
```

Everyday analogy

A normal instruction is like a teacher who corrects one exam paper at a time. SIMD is like a teacher who puts 8 papers side by side and corrects question 1 on all of them with one move. This is why it is used in image and video processing, games, and artificial intelligence, where we apply the same operation to millions of numbers.

4.6 The instruction cycle and the pipeline

4.6.1 The five stages

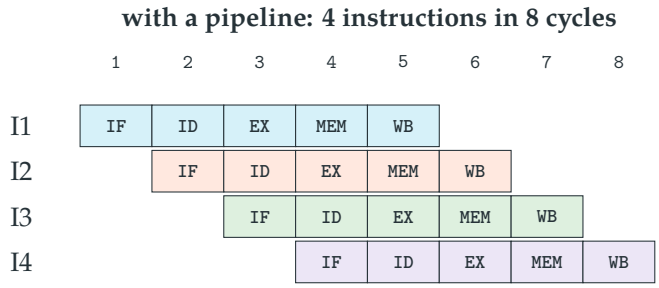
In modern processors, each instruction is split into stages, and each stage has its own circuit:

Stage	Name	Work
IF	fetch	read the instruction from memory (address in RIP)
ID	decode	understand the instruction and read the registers
EX	execute	compute the result in the UAL, or compute an address
MEM	memory	read or write memory if needed
WB	write back	store the result in the register

4.6.2 Without a pipeline and with a pipeline

Everyday analogy

In a laundry: washing takes one hour, drying one hour, ironing one hour. To wash 3 bags one after the other we need 9 hours. But when the first bag goes from the washing machine to the dryer, we put the second bag in the washing machine at once. The three machines work together, so we finish in only 5 hours. This is a pipeline.



Definition

For n instructions and k stages (one cycle per stage):

- without a pipeline: $n \times k$ cycles.
- with a pipeline: $k + (n-1)$ cycles: the first one needs k cycles, then one instruction finishes in **every** cycle.

Step by step: 100 instructions in a 5-stage pipeline

1. Without a pipeline: $100 \times 5 = 500$ cycles.
2. With a pipeline: $5 + (100 - 1) = 104$ cycles.
3. The speed-up: $\frac{500}{104} \approx 4.8$ times, close to the number of stages (5).

4.6.3 Hazards: when does the pipeline stop?

1. **Data hazard**: `add rax, rbx` then `add rcx, rax`. The second instruction needs the new RAX, but the first one has not written it yet. The fix: the processor "forwards" the result directly from EX to the second instruction, or waits one cycle.
2. **Control hazard**: after `jmp` or `jne`, the instructions fetched after it may be wrong. The fix: **branch prediction** (Fr. *prédiction de branchement*): the processor guesses if it will jump, and if it guessed wrong, it throws away the wrong instructions and starts again.
3. **Structural hazard**: two instructions need the same circuit in the same cycle (like memory).

4.6.4 Example: `mov`, `add` and `jmp` in 64 bits

We follow three instructions from the program `same.asm` and the programs of this chapter, with their real bytes:

Stage	`mov rax, 60`	`add rax, rbx`	`jmp fin`
IF	fetch 7 bytes 48 C7 C0 3C 00 00 00	fetch 3 bytes 48 01 D8	fetch 2 bytes EB 00
ID	REX.W: 64 bits, destination RAX, the value 60 is in the instruction	64 bits, source RBX, destination RAX; read both registers	short jump, offset 0
EX	nothing to compute (the value is ready)	UAL: RAX + RBX, and compute the flags	compute the target: RIP + 0
MEM	nothing	nothing	nothing
WB	RAX ← 60	RAX gets the result; update RFLAGS	RIP gets the target; throw away wrongly fetched instructions

Did you know?

The Core i7 we used does much more than a 5-stage pipeline: it has more than 14 stages, and it can start **4 instructions in every cycle**. It even runs them **out of order** when they do not depend on each other, then puts the results back in order. All this happens without the programmer seeing any difference.

4.7 Exercises

Exercise 4.1

How many bytes can a processor address with a 36-bit address? (This is what the PAE extension allowed in 32-bit processors.)

Solution

$2^{36} = 2^6 \times 2^{30} = 64$ GB.

Exercise 4.2

RAX = FFFFFFFFFFFFFFFF$_\text{H}$. What is RAX after: (a) `mov eax, 1` (b) `mov ax, 1` (c) `mov al, 1`?

Solution

(a) 0000000000000001$_\text{H}$ (writing 32 bits clears the high half).
(b) FFFFFFFFFFFF0001$_\text{H}$. (c) FFFFFFFFFFFFFF01$_\text{H}$.

Exercise 4.3

An array of `int` (4 bytes each) starts at 601000$_\text{H}$. What is the address of element number 10? Write the addressing in NASM.

Solution

601000$_\text{H}$ + 10 × 4 = 601000$_\text{H}$ + 40 = 601028$_\text{H}$. The writing: `[tab + rcx*4]` with RCX = 10.

Exercise 4.4

A 6-stage pipeline, and 1000 instructions. Compute the number of cycles without and with a pipeline, and the speed-up.

Solution

Without: $1000 \times 6 = 6000$. With: $6 + 999 = 1005$. Speed-up $= \dfrac{6000}{1005} \approx 5.97$.

Exercise 4.5

In this program, where is the data hazard? `mov rax, [x]`, then `add rax, 5`, then `mov [y], rax`.

Solution

The second instruction needs the RAX that the first one reads from memory, and the third one needs the RAX that the second one computes: each instruction depends on the one just before it.

Chapter 5

Modern Processor Architectures

Summary: what will we learn in this chapter?

- How processors grew, and why the frequency stopped going up.
- Multi-core processors and parallel processing: the benefits and the problems, and Amdahl's law.
- Cache memory and the memory hierarchy, and why the order in which we read data matters so much.

Watch out

All the numbers in this chapter were **really measured** on a laptop with an Intel Core i7-8665U processor (4 cores, 8 threads, L1 cache = 32 KB and L2 = 256 KB per core, and a shared L3 = 8 MB). If you repeat the tests on your computer, your numbers will be a little different, but the effects will be the same.

5.1 Modern processors: trends and changes

5.1.1 Fifty years of progress

Processor	Year	Transistors	Frequency	Cores
Intel 4004	1971	2 300	0.74 MHz	1
Intel 8086	1978	29 000	5–10 MHz	1
Intel 80386	1985	275 000	16–33 MHz	1
Pentium 4	2000	42 million	1.3–3.8 GHz	1
Core i7-8665U	2019	about 2 billion	1.9–4.8 GHz	4
processors of 2024	2024	tens of billions	about 5 GHz	from 8 to more than 100

5.1.2 The frequency wall

Until about 2004, each year brought a faster processor because the **frequency** went up. Then the frequency stopped at about 4 to 5 GHz. The main reason is **heat**:

- Every clock pulse uses energy. The higher the frequency, the more energy is used, and energy turns into heat.
- At about 4 GHz the processor gets so hot that a normal computer cannot cool it.

Everyday analogy

A runner runs faster and faster, until he reaches a speed he cannot keep without falling. Instead of asking him to run faster, we add other runners who share the distance with him. This is exactly what processor makers did.

5.1.3 Today's trends

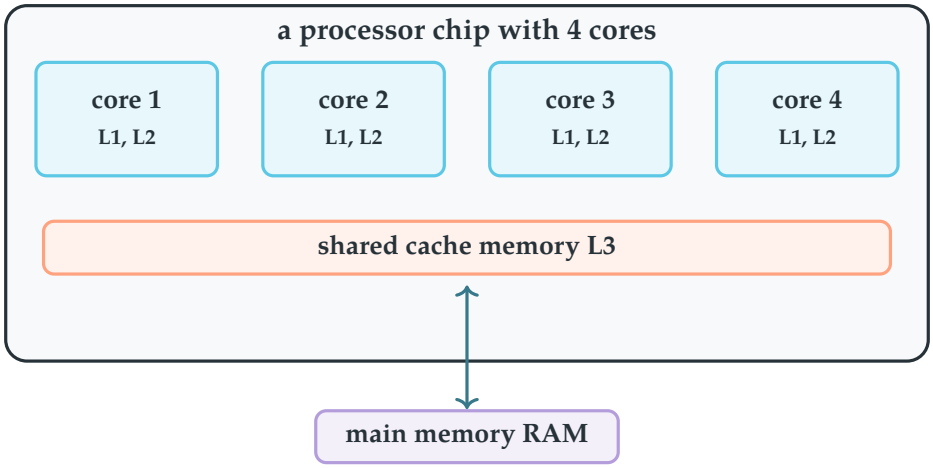
1. **More cores** instead of a higher frequency (next section).
2. **Bigger and smarter cache memory**, because memory became much slower than the processor (last section).
3. **Special instructions**: SIMD (Chapter 4), and speed-up for artificial intelligence and encryption.
4. **Different cores on the same chip**: strong cores for heavy work and economical cores for light work, to save battery.

5.2 Multi-core processors and parallelism

5.2.1 The ideas

Definition

- A **core** (Fr. *cœur*): a complete processor (control unit, arithmetic unit, registers, L1 cache).
- A **multi-core processor** (Fr. *processeur multi-cœur*): several cores on one chip.
- A **thread** (Fr. *thread, fil d'exécution*): a series of instructions that runs on its own. A program may have one thread or several.
- **Parallel processing** (Fr. *traitement parallèle*): running several parts of the work **at the same moment** on different cores.



Everyday analogy

A kitchen with one cook (one core) prepares one dish after another. A kitchen with 4 cooks (4 cores) prepares 4 dishes at the same time. But they share the same fridge (L3 and RAM), and one of them may wait for his turn in front of it. And one complex dish only gets faster if we can split it into independent tasks.

5.2.2 Hyper-Threading: one core for two threads

Simultaneous multithreading (Fr. *multithreading simultané*) (SMT, or Hyper-Threading) makes one core look like two cores: when a thread waits for data from memory, the second thread uses the free arithmetic units. This is why our processor shows 8 "logical processors" even though it has only 4 cores. But if the two threads need the same arithmetic unit all the time, we gain almost nothing, as we will see in the test.

5.2.3 A test: the same work on 1, 2, 4 and 8 threads

The program below computes the sum of the square roots of 400 million numbers, and shares the work between k threads:

```c
/* The same work shared between 1, 2, 4 and 8 threads */
#include <stdio.h>
#include <math.h>
#include <pthread.h>
#include <time.h>

#define WORK 400000000L
static int k;                       /* number of threads */
static double part[8];              /* result of each thread */

static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec * 1e-9;
}

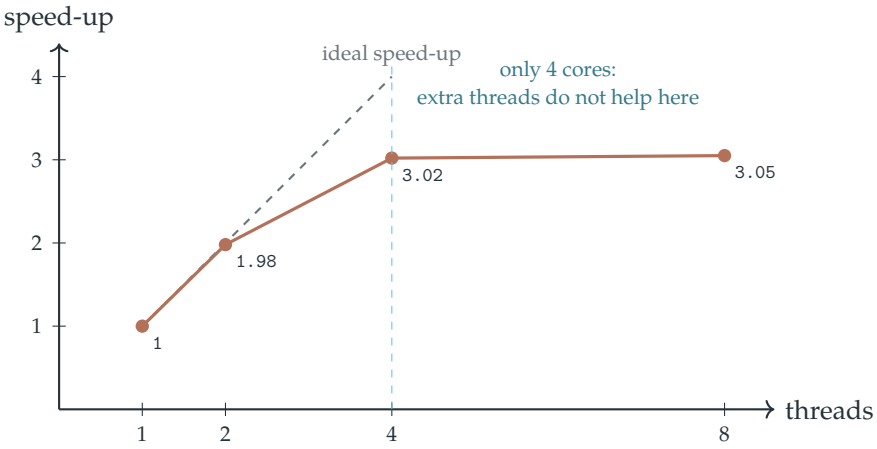
static void *worker(void *arg)      /* thread id computes its part */
{
    long id = (long) arg;
    double s = 0;
    for (long i = id; i < WORK; i += k)
        s += sqrt((double) i);
    part[id] = s;
    return NULL;
}

int main(void)
{
    double t1 = 0;
    for (k = 1; k <= 8; k *= 2) {
        pthread_t th[8];
        double t = now(), total = 0;
        for (long id = 0; id < k; id++)
            pthread_create(&th[id], NULL, worker, (void *) id);
        for (int id = 0; id < k; id++) {
            pthread_join(th[id], NULL);
            total += part[id];
        }
        t = now() - t;
        if (k == 1) t1 = t;
        printf("%d thread(s): %.3f s  speedup %.2f\n", k, t, t1 / t);
    }
    return 0;
}
```

screen

```
1 thread(s): 0.224 s  speedup 1.00
2 thread(s): 0.113 s  speedup 1.98
4 thread(s): 0.074 s  speedup 3.02
8 thread(s): 0.073 s  speedup 3.05
```

Line	What does it do?
18–26	Each thread takes the numbers id, $id + k$, $id + 2k$...so it computes a part of the sum that does not overlap the others.
34–35	`pthread_create`: start k threads; each one runs on a core.
36–39	`pthread_join`: wait for each thread to finish, then add the partial results.
40–42	Speed-up = time with one thread ÷ time with k threads.



Step by step: how do we read these results?

1. **2 threads**: speed-up 1.98, very close to 2. The work was split into two fully independent halves.
2. **4 threads**: speed-up 3.02, not 4. When the four cores work together the processor gets hot, so it lowers its frequency by itself. And a part of the program (starting the threads and adding the results) is not parallel.
3. **8 threads**: speed-up 3.05, almost the same as 4 threads. Our processor has only 4 real cores, and the extra threads share the same arithmetic units, so the work does not go faster.

5.2.4 Amdahl's law: the limits of parallelism

Most programs have a part that **cannot be shared** (reading the file, showing the result, waiting for the user...).

Definition

If the parallel part of the program is p (between 0 and 1), and we have n cores, the maximum speed-up is:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

Step by step: a program that is 90% parallel

$p = 0.9$, and the serial part $1 - p = 0.1$.

1. With 4 cores: $S = \dfrac{1}{0.1 + \frac{0.9}{4}} = \dfrac{1}{0.1 + 0.225} = \dfrac{1}{0.325} \approx 3.08$.
2. With 16 cores: $S = \dfrac{1}{0.1 + 0.05625} \approx 6.4$.
3. With an infinite number of cores: $\dfrac{p}{n} \to 0$, so $S = \dfrac{1}{0.1} = 10$.

However many cores we add, the program will never be more than 10 times faster, because the 10% serial part stays the same.

Did you know?

Notice that Amdahl's law with $p = 0.9$ and 4 cores gives 3.08, very close to the speed-up of 3.02 we measured! A simple law explains reality well.

5.2.5 Benefits and challenges

Benefits	Challenges
more performance without raising the frequency	the program must be written with several threads, which is harder
less energy for the same work	the serial part limits the speed-up (Amdahl's law)
several programs run smoothly at the same time	**synchronization**: two threads writing the same variable can break the result
if one task fails, the other cores keep working	**cache coherence**: all the cores must see the same value of a variable

Example: the synchronization problem

Two threads run `x = x + 1` on the same variable, and x = 5 at the start:

1. Thread 1 reads x = 5.
2. Thread 2 also reads x = 5 (before the first one writes).
3. Thread 1 computes $5 + 1$ and writes 6.
4. Thread 2 computes $5 + 1$ and also writes 6.

The result is 6 instead of 7: one increase is lost! This is why we use **locks** (mutex) or **atomic** instructions (`lock add` in x86) that stop two threads from changing the variable at the same time. In our program we avoided the problem: each thread writes into its own box `part[id]`.

5.3 Cache memory and the memory hierarchy

5.3.1 The problem: memory is very slow

The processor can run several instructions every nanosecond, but reading the main memory takes about 100 nanoseconds. If the processor waited for memory every time, it would do nothing most of the time.

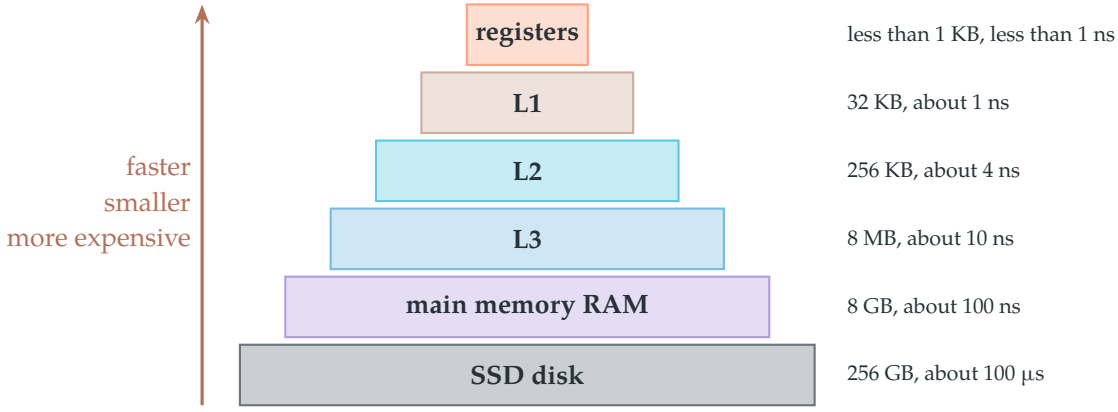
Definition

Cache memory (Fr. *mémoire cache, antémémoire*): a small and very fast memory (SRAM) close to the processor. It keeps a **copy** of the data used recently, or of the data we will need soon.

Everyday analogy

When you revise for an exam, you do not put all the books of the library on your desk. On the desk you put the books you need now (L1), in your bag the books of the week (L2), in your cupboard at home the books of the year (L3), and the rest stays in the far library (RAM). The closer the place, the smaller and the faster it is.

5.3.2 The memory hierarchy



5.3.3 A test: we can see the memory levels

The program below reads an array in a random order (each read gives the address of the next read, so the processor cannot guess), and measures the average time of one read, for different array sizes:

```c
/* Average time of one memory access, for different data sizes.
   Each access depends on the previous one (pointer chasing),
   so the processor cannot guess the next address. */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

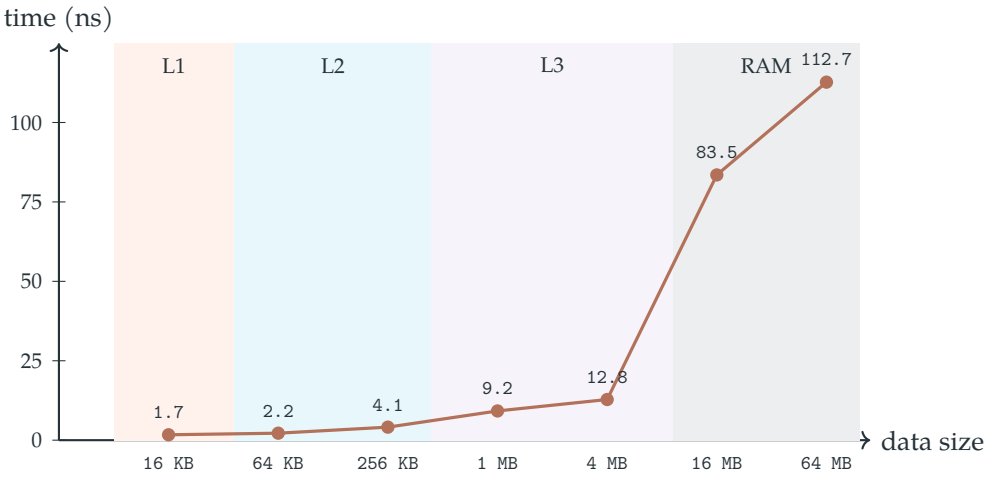
static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec * 1e-9;
}

int main(void)
{
    for (long kb = 16; kb <= 65536; kb *= 4) {
        long n = kb * 1024 / sizeof(long);
        long *next = malloc(n * sizeof(long));
        long *order = malloc(n * sizeof(long));
        for (long i = 0; i < n; i++) order[i] = i;
        for (long i = n - 1; i > 0; i--) {          /* random order */
            long j = rand() % (i + 1), t = order[i];
            order[i] = order[j]; order[j] = t;
        }
        for (long i = 0; i < n; i++)                /* one big cycle */
            next[order[i]] = order[(i + 1) % n];

        long p = 0, steps = 20000000;
        double t0 = now();
        for (long s = 0; s < steps; s++)
            p = next[p];
        double ns = (now() - t0) / steps * 1e9;
        printf("%6ld KB : %5.1f ns%s\n", kb, ns, p < 0 ? "!" : "");
        free(next); free(order);
    }
    return 0;
}
```

screen

```
    16 KB :   1.7 ns
    64 KB :   2.2 ns
   256 KB :   4.1 ns
  1024 KB :   9.2 ns
  4096 KB :  12.8 ns
 16384 KB :  83.5 ns
 65536 KB : 112.7 ns
```



Step by step: what do we see in the curve?

1. **Up to 32 KB**: the whole array is in L1, and a read takes only 1.7 ns.
2. **64 KB and 256 KB**: the array does not fit in L1 any more, so we read from L2: the time goes up to 2 then 4 ns.
3. **1 MB and 4 MB**: we read from the shared L3: about 9 to 13 ns.
4. **16 MB and more**: the array is bigger than L3 (8 MB), so we go to RAM: the time jumps to 84 then 113 ns, about **65 times** slower than L1!

The steps in the curve fall exactly at the cache sizes of our processor.

5.3.4 The principle of locality: why does cache memory work?

Cache memory is small, but it usually holds what we need, thanks to two natural habits of programs:

- **Temporal locality** (Fr. *localité temporelle*): the data we used now will often be used **again soon** (the loop counter, the sum...).
- **Spatial locality** (Fr. *localité spatiale*): after reading an address, we often read the **nearby addresses** (array elements, instructions that follow each other).

Definition

A **cache line** (Fr. *ligne de cache*): the cache does not move one byte, but a whole block of **64 bytes** next to each other. When we read `t[0]`, `t[1]` to `t[15]` (for 4-byte numbers) come with it for free.

5.3.5 A test: the order of reading changes everything

```c
/* Same sum, two orders: row by row, or column by column */
#include <stdio.h>
#include <time.h>

#define N 4096
static int m[N][N];                 /* 4096 x 4096 ints = 64 MB */

static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec * 1e-9;
}

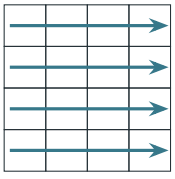
int main(void)
{
    long s = 0;
    double t;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            m[i][j] = 1;

    t = now();
    for (int i = 0; i < N; i++)      /* row by row: neighbours */
        for (int j = 0; j < N; j++)
            s += m[i][j];
    printf("rows    : %.3f s\n", now() - t);

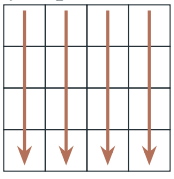
    t = now();
    for (int j = 0; j < N; j++)      /* column by column: jumps */
        for (int i = 0; i < N; i++)
            s += m[i][j];
    printf("columns : %.3f s\n", now() - t);
    printf("sum = %ld\n", s);
    return 0;
}
```

```
screen
rows    : 0.009 s
columns : 0.301 s
sum = 33554432
```

row by row: each read is next to the one before



column by column: each read jumps 16 KB



Step by step: why are columns 33 times slower?

1. In C, the array `m[4096][4096]` is stored **row after row**: `m[0][0]`, then `m[0][1]`, then...
2. **Row by row**: each cache line (64 bytes) holds 16 numbers next to each other. We read from RAM once, then use 16 numbers from the cache. And the processor notices we move forward regularly, so it fetches the next lines in advance.
3. **Column by column**: `m[0][j]` and `m[1][j]` are $4096 \times 4 = 16\,384$ bytes apart. Every read is in a new cache line, so we use only one number out of the 16 we moved.
4. The real result: 0.009 s against 0.301 s, about **33 times** slower, for exactly the same operations!

5.3.6 Hits, misses and average access time

Definition

- A **hit** (Fr. *succès*): the data is in the cache.
- A **miss** (Fr. *défaut*): the data is not in the cache, so we go to the next (slower) level.
- The **hit rate** h: the share of reads that found their data in the cache.

$$\text{average time} = h \times t_{cache} + (1 - h) \times t_{RAM}$$

Step by step: $t_{cache} = 1$ ns, $t_{RAM} = 100$ ns

1. Hit rate 90%: $0.9 \times 1 + 0.1 \times 100 = 0.9 + 10 = 10.9$ ns.
2. Hit rate 99%: $0.99 \times 1 + 0.01 \times 100 = 0.99 + 1 = 1.99$ ns.

Going from 90% to 99% makes memory **more than 5 times faster**. This is why modern processors work hard to improve the hit rate.

5.3.7 Where do we put data in the cache? (mapping)

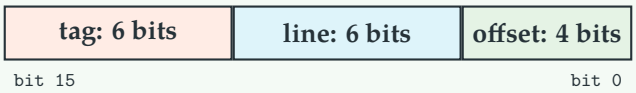
The cache is split into lines. We must decide in which line we put a block coming from memory:

Kind	Rule	Advantage and drawback
direct mapped	each block has only one possible line: block number mod number of lines	simple and fast, but two blocks may fight for the same line
fully associative	any block in any line	fewer conflicts, but searching all the lines is expensive
set associative	the block goes to a given set, and inside it to any of k lines	the middle way, used in all modern processors

Step by step: splitting an address in a direct-mapped cache

A 1 KB cache, with 16-byte lines, and 16-bit addresses.

1. Number of lines: $1024 \div 16 = 64 = 2^6$ lines, so we need **6 bits** for the line number.
2. Inside the line there are $16 = 2^4$ bytes, so **4 bits** for the offset.
3. The rest: $16 - 6 - 4 = 6$ bits, called the **tag**: it tells which memory block is in the line now.



Example: $1A2C_H = 000110\,100010\,1100_2$. Tag $000110_2 = 6$, line $100010_2 = 34$, and offset $1100_2 = 12$.

5.3.8 Replacement and writing

- **Replacement policy**: when a set is full, which line do we throw out? Usually **LRU** (Least Recently Used): the line that has not been used for the longest time.
- **Write policy**:
  - **Write-through**: we write in the cache and in RAM together. Simple but slow.
  - **Write-back**: we write only in the cache, and write in RAM only when the line is thrown out. Faster, and used today.

Did you know?

Remember the Harvard model from Chapter 1: the L1 cache is split into two parts, L1i for instructions and L1d for data (32 KB each in our processor), so the processor can fetch an instruction and read data at the same time. But L2, L3 and RAM are shared by both, as in the Von Neumann model.

5.4 Exercises

Exercise 5.1

A program is 75% parallel. Compute the speed-up with 2, 4 and 8 cores, and the maximum speed-up.

Solution

$1 - p = 0.25$. $S(2) = \dfrac{1}{0.25 + 0.375} = 1.6$, $S(4) = \dfrac{1}{0.25 + 0.1875} \approx 2.29$, $S(8) = \dfrac{1}{0.25 + 0.09375} \approx 2.91$. Maximum $= \dfrac{1}{0.25} = 4$.

Exercise 5.2

A cache with a time of 2 ns, a main memory of 80 ns, and a hit rate of 95%. What is the average time?

Solution

$0.95 \times 2 + 0.05 \times 80 = 1.9 + 4 = 5.9$ ns.

Exercise 5.3

A direct-mapped cache: 4 KB, 32-byte lines, 32-bit addresses. How many bits for the offset, the line and the tag?

Solution

Offset: $32 = 2^5$, so 5 bits. Lines: $4096 \div 32 = 128 = 2^7$, so 7 bits. Tag: $32 - 7 - 5 = 20$ bits.

Exercise 5.4

Explain why this code is slow, and how to fix it:

```
for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
        s += m[i][j];
```

Solution

It reads the array column by column, so every read is in a different cache line (no spatial locality). We fix it by swapping the two loops: `for (i ...) for (j ...)`, so we read row by row.

Exercise 5.5

Why did the program `threads.c` not get better when going from 4 to 8 threads on our processor?

Solution

Because the processor has only 4 real cores. Threads 5 to 8 run with Hyper-Threading on the same cores, and the program uses the arithmetic unit all the time, so there are no free units for the second thread to use.

Appendix A

8086 Instruction Summary

This appendix puts in one place all the instructions we studied in Chapter 3, so you can find them quickly while you write your programs. Symbols: d = destination, s = source, r = register, m = memory box.

The most used DOS functions (`int 21h`)

AH	Function	Inputs / outputs
01_H	read a key and show it	output in AL
02_H	print a character	the character in DL
09_H	print a string ending with `$`	the address in DX
$0A_H$	read a line into a buffer	the address in DX
$4C_H$	end the program	the exit code in AL

The instructions

Instruction	Meaning	Flags changed
Transfer		
`mov d, s`	d ← s	none
`xchg a, b`	swap a and b	none
`lea r, m`	r gets the address of m	none
`push s`	SP ← SP - 2, then [SP] ← s	none
`pop d`	d ← [SP], then SP ← SP + 2	none
`in al, port`, `out port, al`	read from a port, write to a port	none
Arithmetic		
`add d, s`	d ← d + s	all
`adc d, s`	d ← d + s + CF	all
`sub d, s`	d ← d - s	all
`sbb d, s`	d ← d - s - CF	all
`inc d`	d ← d + 1	all except CF
`dec d`	d ← d - 1	all except CF
`neg d`	d ← -d	all
`cmp a, b`	computes a - b without keeping it	all
`mul s` (8 bits)	AX ← AL * s	CF, OF
`mul s` (16 bits)	DX:AX ← AX * s	CF, OF
`div s` (8 bits)	AL ← AX / s, AH ← AX mod s	undefined
`div s` (16 bits)	AX ← DX:AX / s, DX ← DX:AX mod s	undefined
Logic and shifts		
`and`, `or`, `xor`	bit by bit operations	CF = OF = 0, ZF, SF
`not d`	flip all the bits	none
`test a, b`	`and` without keeping the result	like `and`
`shl d, n`, `shr d, n`	shift: multiply or divide by 2^n	CF = last bit out
`rol d, n`, `ror d, n`	rotate	CF
Flow control		
`jmp L`	always jump	none
`je`, `jne`, `jz`, `jnz`	depends on ZF	–
`jg`, `jge`, `jl`, `jle`	**signed** comparison	–
`ja`, `jae`, `jb`, `jbe`	**unsigned** comparison	–
`loop L`	CX ← CX - 1, then jump if CX $\neq 0$	none
`jcxz L`	jump if CX = 0	–
`call P`, `ret`	call a procedure, come back from it	none
`int n`, `iret`	interrupt, come back from it	IF = TF = 0 / restored
Strings		
`movsb`, `movsw`	ES:[DI] ← DS:[SI], then move SI and DI forward	none
`rep`	repeat the instruction CX times	–
`cld`, `std`	DF = 0 (forward), DF = 1	DF

Appendix B

EMU8086 and NASM: differences in writing

The programs of Chapter 3 are written for EMU8086 (a syntax close to MASM), and the programs of Chapter 4 for NASM. The processor is the same, but the two assemblers do not write some things the same way:

What we want	EMU8086 / MASM	NASM
address of a variable in a register	`lea dx, msg` or `mov dx, offset msg`	`mov dx, msg`
value of a variable	`mov al, x` or `mov al, [x]`	`mov al, [x]` (brackets needed)
giving the size	`mov byte ptr [bx], 5`	`mov byte [bx], 5`
a hex number	`0FFh`	`0xFF` or `0FFh`
a procedure	`p proc` ... `p endp`	a simple label `p:`
a constant	`N equ 10`	`N equ 10`
an array of 10 zeros	`t db 10 dup(0)`	`t times 10 db 0`
assembling	the *Emulate* button	`nasm -f bin prog.asm -o prog.com`

Watch out

A good habit: always write the brackets `[ ]` when you mean the content of memory, even in EMU8086. Your program stays clear, and works with both assemblers with few changes. This is what we did in all the programs of this course.

Appendix C

Glossary

English	Français
computer	ordinateur
processor, CPU	processeur, UCT
central unit	unité centrale
main memory	mémoire centrale
volatile memory	mémoire volatile
bus	bus
address bus / data bus / control bus	bus d'adresses / de données / de contrôle
register	registre
program counter (PC)	compteur ordinal (CO)
instruction register (IR)	registre d'instruction (RI)
memory address register (MAR)	registre d'adresse mémoire (RAM)
memory data register (MDR)	registre d'information mémoire (RIM)
accumulator	accumulateur
flag	indicateur, drapeau
arithmetic logic unit (ALU)	unité arithmétique et logique (UAL)
control unit	unité de commande (UC)
decoder / sequencer	décodeur / séquenceur
half adder / full adder	demi-additionneur / additionneur complet
two's complement	complément à deux
instruction set	jeu d'instructions
opcode	code opération
addressing mode	mode d'adressage
effective address / physical address	adresse effective / adresse physique
segment, offset	segment, déplacement
instruction cycle	cycle d'instruction
fetch, decode, execute	recherche, décodage, exécution
assembly language / assembler	langage d'assemblage / assembleur
stack	pile
procedure, subroutine	procédure, sous-programme
interrupt / interrupt vector table	interruption / table des vecteurs d'interruption
I/O port	port d'entrée/sortie
protected mode	mode protégé
flat memory model	modèle mémoire plat
pipeline / branch prediction	pipeline / prédiction de branchement
core / thread	cœur / thread, fil d'exécution
cache memory / cache line	mémoire cache / ligne de cache
hit / miss	succès / défaut
temporal and spatial locality	localité temporelle et spatiale

References

1. A. S. Tanenbaum, *Structured Computer Organization*, Pearson, 2012.
2. D. A. Patterson, J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
3. P. Zanella, Y. Ligier, E. Lazard, *Architecture et technologie des ordinateurs*, 6e édition, Dunod, 2018.
4. B. B. Brey, *The Intel Microprocessors: 8086/8088 … Core2 with 64-bit Extensions*, Pearson.
5. EMU8086, Microprocessor Emulator.
6. NASM, The Netwide Assembler: https://www.nasm.us/
7. MASM, Microsoft Macro Assembler reference: https://learn.microsoft.com/en-us/cpp/assembler/masm/
8. GAS, The GNU Assembler: https://sourceware.org/binutils/docs/as/